

The Ecosystem Engine

Retention and game integrity for an online poker operator — built on **21,605,687** real hands of recorded play.

THE FINDING, IN ONE SENTENCE

\$604,163 of next week's rake sits with players who are more likely than not to stop playing — **30%** of it in the recreational segment — and contacting the top 10% of the list by expected loss (**7,727** players) puts **\$421,064** of it back in reach.

Weekly rake in view **\$2,114,626**, of which **34.4%** is measured against the pot and the rest priced from what comparable hands actually paid · five venues (iPoker excluded from every money question) · July 2009, **2009 US dollars**, no currency conversion and no inflation adjustment.

RECORDS ENGINEERED

116.6M

seat-rows joined and aggregated into a 21 MB modelling table

MODELS BUILT IN MLLIB

4

GBT · random forest · logistic regression
· K-means, against three baselines

HANDS RECONCILED EXACTLY

100%

every hand parsed or dropped with a recorded reason — total loss 0.23%

SUBMITTED BY

TEAM: add names & roll numbers

Contribution split is recorded in Appendix C.

ENGAGEMENT ARTIFACTS

- This report · deliverables/report/
- Executive deck, 10 slides · deliverables/presentation/
- Executable notebook · notebooks/
- Decision dashboard · docs/dashboard.html

Contents

NINE SECTIONS, AS THE BRIEF NAMES THEM

1	Executive Summary	The problem, the finding and the four decisions — on one page
2	Business Context	How a poker room earns, why the ecosystem is the asset, and the KPIs that follow
3	Data Understanding — the 5 Vs	Volume, variety, velocity, veracity and value, each answered with a measurement
4	Enterprise Architecture	The medallion lake, the compute, and what happens at ten times the scale
5	Data Engineering	Parsing, the five defects that killed runs, missing values, outliers, feature engineering (5.6) and the rake derivation
6	Machine Learning	Choosing the question, three MLlib approaches, and how each was evaluated
7	Business Insights	Five findings, each with a number attached
8	Strategic Recommendations	Four costed decisions with owners, the risks, and a 90-day roadmap
9	Appendix	References · AI-usage disclosure · team contributions · outcome mapping · reproducibility · glossary

HOW TO READ THE NUMBERS IN THIS REPORT

Every figure sits in a tagged span carrying the path of the JSON key it came from, and `src/report_figures.py` rewrites all of them from the model runs' own output files before the PDF is exported. A run that changes nothing is the proof that this document still matches the analysis. **No number in this report was typed by hand.**

FIVE THINGS WE DO NOT HIDE

- The data is from **July 2009**. We argue the method transfers; we never bury the date.
- Rake is **derived**, never a recorded column — 34.4% measured.
- Twenty-six days of data. This is **short-horizon lapse**, not churn.
- There is **no ground truth** for who is a bot, so we never claim to detect one.
- Player IDs are per venue. The same human on two sites is two rows, **never merged**.

THE ENGAGEMENT IN ONE LINE

Business problem → data sources → ingestion → distributed storage → data engineering → feature engineering → machine learning → business insights → **executive recommendations**

Sections 2–8 follow this spine in order. Section 5.6 and Section 6 carry the two phases the rubric weights most heavily.

1 Executive Summary

WRITTEN TO BE THE ONLY PAGE ANYONE READS

The problem

An online poker operator never bets. It takes a small percentage of each pot — the **rake** — and it earns nothing at all when no hand is dealt. Measured on every venue whose money reconciles, the median rake is **4.3%–5.0%** of the pot, and **exactly \$0.00 when no flop is dealt**. Revenue is therefore a pure function of how many hands get played, which is a pure function of how many losing players are willing to keep sitting down. A small population of high-volume professionals wins that money steadily; the recreational players who supply it get discouraged and stop logging in — quietly, without complaining, and without cancelling anything. When the supply stops, the games stop, and the rake dies with them. **This is a retention and game-integrity problem, not a game-AI problem.**

What we built

We rebuilt **21,605,687** published hands of real-money play into a medallion data lake (**16.3** GB of raw text → **2.3** GB of Parquet → a **21** MB modelling table), then trained three PySpark MLlib approaches on it: **K-means segmentation** to say what kind of player each account is, **gradient-boosted classification** to say who stops playing next week, and a **money layer** that prices every hand's rake and attributes it to the players who paid it. Every model is scored against a baseline a manager could build in a spreadsheet, because a score with nothing beside it is not evidence.

What we found

\$604,163 of next week's rake sits with players who are more likely than not to play zero hands, and **\$421,064** of it — **70%** — is reachable with **7,727** calls, if the list is sorted the right way.

- **We lose the players we earn from, and keep the ones we don't.** The two segments that lose money fastest are the two that leave fastest: the Recreational player loses -14.3 bb/100 and lapses at 77.1%; the Gambler loses -31.8 and lapses at 74.6%. The professional Grinder loses least (-3.3, essentially just the rake) and is the **stickiest** account on the platform.
- **Going quiet is predictable a week ahead.** Gradient-boosted trees reach **0.857** ROC-AUC on 22,596 held-out players, from behaviour recorded strictly before the week being predicted.
- **A retention list sorted by churn risk is almost exactly the wrong list.** Ranking the same players by risk alone and by expected loss produces two lists that share **0.1%** of their names. The risk-only list is *better* at finding leavers — 7,554 of them at 98% precision — and it reaches **0.8%** of the money. It is a list of certain departures worth nothing.
- **Two thirds of the operator's own revenue is not written down anywhere.** Rake had to be derived from the pot; only 34.4% of it reconciles exactly. That is a data-governance finding as much as an analytical one.

What management should do

#	DECISION	THE NUMBER	OWNER
1	Rank Monday's call list by expected loss, not by churn risk The ranking rule is the decision; the model is only an input to it	\$421,064 reached instead of \$4,751	Retention & CRM
2	Spend the retention budget on the recreational seat, not the top spender The professionals are paid out of recreational deposits, not out of the operator	56% of the risk \$340,379	VIP & promotions
3	Run the list per venue, never pooled One venue is 45% of the exposure and has the thinnest evidence behind it	lapse 40.4%–77.9% coverage 11%–97%	Regional operations
4	Record rake at source, and this stops being an estimate One logged column per hand retires the largest caveat in this report	65.6% modelled today target: 100% measured	Data engineering

Scope and limits. Figures are in 2009 US dollars at stakes from 25NL to 1000NL, over a 26-day observation window; "leaving" means zero hands in the next seven days, measured against each venue's own last recorded day, and is called **short-horizon lapse** throughout rather than churn. The full limitations are in §6.7 and §8.2, and every one of them is raised by us rather than left to be found.

2.1 The operator does not gamble — it rents the seat

Every other casino product carries house risk: the operator wins when the customer loses. Poker does not work that way. Players play against each other, and the operator takes a percentage of each pot — capped, and charged only once a flop is dealt. The industry phrase for the cap is “no flop, no drop”, and it is visible in our own data: across the 2,473,612 hands that ended before a flop and whose money reconciles, the derived rake is **exactly \$0.00** on **99.95%** of them — 2 of the five venues score a clean 100.000%. The operator’s revenue is therefore not a function of who wins. It is a function of **how many pots are contested, at what size, by how many people**.

That makes an online poker room an unusual business to run: its product is not a game, it is a **liquid marketplace**. It has to keep enough players seated, at enough tables, at enough stake levels, for a game to start when someone logs in. Liquidity is the asset, and liquidity is a population, not a balance sheet item.

2.2 The ecosystem, and how it fails

WHERE THE MONEY ACTUALLY GOES — AND WHY THE OPERATOR IS NOT ON EITHER SIDE OF IT



The operator is a landlord in a room where one group of tenants is quietly being emptied by another. Both bb/100 figures and the table count are cluster means measured in §6.3, not assumptions.

The failure mode is what makes this hard to see from a revenue dashboard. A recreational player does not close their account, complain, or trigger any event the business records. They simply stop logging in. Revenue does not fall the day they leave — it falls weeks later, when the tables they used to fill will not start. By then the diagnosis is “the games got tougher”, which is a symptom, not a cause.

2.3 Why this is a big-data problem and not a spreadsheet one

The behaviour that predicts a quiet departure is not in any summary table. It is in the sequence of individual actions — how often someone voluntarily put money in, how many tables they had open, whether their volume was decaying into last Tuesday. To see it you have to work at the level of the hand and the seat, and there are 116,621,636 seat-rows and 183,671,936 action-rows in this dataset. Excel stops at 1,048,576 rows: the seat table is 111× that limit and the action table 175×. Distributed processing here is not a technology preference; it is the only way the question can be asked at all.

2.4 From an ambiguous business problem to an analytical one

“Our ecosystem is unhealthy” is not a question a model can answer. Translating it took three narrowing steps, and each one is a decision the business could have made differently.

#	THE AMBIGUOUS VERSION	THE ANALYTICAL VERSION WE COMMITTED TO
1	“Which players matter?”	Cluster every account on prior-window behaviour, volume and multi-tabling, and let the groups fall out of the data rather than out of our assumptions. Unsupervised, K-means. §6.3
2	“Who is about to churn?”	Twenty-six days cannot show that anyone left for good, so we predict a label the data can actually carry: zero hands in the next seven days , measured against each venue’s own last recorded day. Supervised classification. §6.2
3	“How much is that worth?”	Price every hand’s rake, attribute it to the players who built the pot, and multiply by a calibrated probability. Expected loss in dollars, per named account. §6.4

2.5 The KPIs this engagement is accountable to

Four measures, chosen because each one can be read off the shipped table and each one moves a budget rather than a chart.

KPI 1 · RAKE AT RISK (WEEKLY)

\$604,163

The sum, over every player, of calibrated lapse probability × that player's weekly rake. It is the single number the dashboard opens on and the one the recommendations move.

KPI 2 · REACH AT A FIXED CONTACT BUDGET

70%

Share of that money sitting inside the top 7,727 accounts the retention team can actually call. It measures the *ranking rule*, not the model.

KPI 3 · RECREATIONAL SHARE OF EXPOSURE

30.2%

The health of the ecosystem, stated in money. If retention spend is working, this share should fall while total liquidity holds.

KPI 4 · MEASURED SHARE OF REVENUE

34.4%

A governance KPI, not an analytical one: how much of the operator's own revenue is recorded rather than reconstructed. Today two thirds is reconstructed.

2.6 Scope, ethics and what we deliberately did not build

A RETENTION MODEL AND A PLAYER-PROTECTION MODEL ARE THE SAME LIST

Nothing in this project models how to win at poker, and nothing identifies a player to exploit. The accounts the model surfaces are the ones **losing money fastest and disengaging quietest** — which is precisely the population a responsible-gaming team would want flagged. We rank who to contact and protect; we never rank who to target. Two capabilities that this data could support and that we ruled out on purpose: **seat-selection advice** (telling a professional which table to join) and **bot detection**, the second because there is no ground truth for it anywhere in the dataset and an unvalidated accusation is worse than no model. Any deployment of this list should carry a stake-down or cool-off option alongside the bonus offer, so the same score cannot only ever be used to sell.

THE DATE, RAISED BY US RATHER THAN FOUND BY YOU

These hands were dealt in **July 2009**. Three structures of the business have not changed since: the operator still earns a capped percentage of contested pots rather than a share of player losses; a minority of high-volume multi-tablers still extracts from a majority of casual players; and a casual player who stops enjoying the game still leaves without saying so. Those are what the model is built on. What is *not* transferable is the magnitude — **\$2,114,626 a week is a 2009 micro-stakes market**, and we neither convert it to rupees (a 2009 exchange rate would have to be invented) nor adjust it for inflation. Read the dollars as a scale, and the method as the deliverable.

Why this dataset. The engagement needed a real, citable, licensed source with player-level identity that survives across files — verified before any modelling, the same player code recurs across consecutive files with **91.7% overlap** within a venue, which is the property the entire project depends on. Full citation in Appendix A.

3

Data Understanding — the 5 Vs

EACH V ANSWERED WITH A NUMBER WE MEASURED, NOT A DEFINITION

THE SOURCE

Dataset	uoftcprg/phh-dataset — Poker Hand History format, real-money online cash games. Zenodo DOI 10.5281/zenodo.17136841, v3, CC BY 4.0.
Citation	Kim, Juho. "Recording and Describing Poker Hands." IEEE Conference on Games (CoG), 2024. DOI 10.1109/CoG60054.2024.10645611.
Scope used	data/handhq only — 21,605,687 No-Limit Hold'em hands from 6 commercial venues, stakes 25NL–1000NL. The archive's other 619 M hands are bots playing bots in a research competition and were deliberately excluded.
Format	.phhs files. Each is a valid TOML document holding many hands; one hand is one table with typed keys, so the raw layer is semi-structured , not tabular.

3.1 Volume — measured at every layer, not estimated

LAYER	ROWS / FILES	ON DISKWHAT ONE ROW IS
Bronze · raw	21,782 files	16.31 GB TOML hand histories, kept byte-for-byte as published
Silver · hands	21,556,835 rows	311 MB one row per hand — pot, rake, stake, table, timestamp
Silver · hand_players	116,621,636 rows	1,086 MB one row per seat — 5.41 per hand
Silver · actions	183,671,936 rows	940 MB one row per decision — 8.52 per hand
Gold · player_lapse	90,469 rows	21 MB the modelling table — 61 columns, one row per player

Sizes are the sum of the files' own byte counts in decimal GB, measured by `src/pipeline_facts.py`. Bronze → Silver compresses **7.0x** while adding types, column statistics and predicate push-down.

The volume figure that matters commercially is not the raw size. It is that **116,621,636** seat-rows compress into a **21 MB** modelling table — a **51x** reduction — because the distributed work happens once and never again. **That is the argument for the architecture, stated as a measurement.**

3.2 Variety — six venues that agree on nothing

The six venues are six independent exports of the same game, and the differences are not cosmetic. They drive nearly every engineering decision in §5.

VENUE	HANDS	SHARE	LAST DAY	DISTINCT TABLES	MEDIAN BB	WINNINGS RECONCILE
PokerStars PS	8,293,797	38.5%	20	11,355	\$0.25	26.9%
iPoker IPN	5,996,741	27.8%	26	1 no usable money	\$2.00	0.0%
PartyPoker PTY	3,092,685	14.3%	26	697	\$0.25	14.1%
Ongame ONG	1,631,839	7.6%	24	1,252	\$4.00	97.1%
Full Tilt FTP	1,283,795	6.0%	23	809	\$6.00	26.8%
Absolute ABS	1,257,978	5.8%	20	1,046	\$1.00	62.1%

"Last day" is the venue's own final recorded day of July 2009 — they do not share a calendar. "Distinct tables" of 1 is not a small venue: it is a venue that wrote the data vendor's name into every table field.

- Types vary by value, not by column. TOML infers the type from the literal, so 25 is an integer and 25.0 a float — *within one field, across hands in one file*. Five money fields do this.
- One venue writes string hand IDs ('R5-2483622-63') where the other five write integers, which is enough to abort a Parquet write eight thousand files into a run.
- One venue records no table identity at all — it writes the data vendor's name, 'HandHQ', into all 5,996,741 of its table fields: 1 distinct table against PokerStars' 11,355.
- Stake levels differ by an order of magnitude — the median big blind runs from \$0.25 to \$6.00, so no dollar figure is comparable until it is normalised to big blinds.

3.3 Velocity — batch here, streaming in production

This dataset arrives as a static archive, so the pipeline we built is honestly a **batch** pipeline and we do not dress it up as anything else. But the velocity of the underlying business is real and worth stating, because it decides what the production version of this system looks like.

The source files are consecutive time slices of roughly **2¼ minutes each**, around 330 tables live at any moment, second-level timestamps on every action. At that rate the six venues generate about **829,000** hands and **7,064,000** action events a day — and a live operator sees them as they happen.

WHAT VELOCITY CHANGES, AND WHAT IT DOES NOT

The label is weekly, so the model does not need to be real-time. Scoring once a night is enough for a team that works a call list on Monday. What *would* justify streaming is the ingest: a Kafka topic per venue writing straight into Bronze would let the recency feature — the strongest predictor in §6.2 — be current to the minute rather than the day. We did not build it here because with a fixed archive it would have been theatre. It is on the roadmap in §8.3.

3.4 Veracity — six real defects, all found by running everything

Sampling one file per venue found **none** of the following. Every one surfaced only when all **21,782** files were parsed, and four of them terminated a full run before they were fixed.

#	DEFECT	WHAT IT DOES, AND WHAT IT COST	SCALE
1	Value-dependent TOML types	The same money field is <code>int</code> in one hand and <code>float</code> in the next. Inference guessed from early rows and killed two full runs at the same batch, with two different errors .	5 fields all venues
2	String hand IDs on one venue	Mixed <code>str/int</code> aborts the Parquet write (<code>ComputeError</code>) about 8,700 files in — two hours of work lost.	1.6 M hands Ongame
3	Hands with no hand key	A file-level handler discarded the whole file to protect against one bad hand — ~31,000 good hands lost to save 31. Fixed by catching at the smallest unit that can fail.	31 files Full Tilt
4	Reused hand IDs within a venue	One venue reissues the same id to genuinely different hands, so the natural join key is not unique . Undetected, it silently fans out a 116 M-row join.	147 ids 547 rows
5	Infinite starting stacks	<code>inf</code> is a legal TOML float, so it parsed silently and reached the feature table intact, where it detonates any scaler or distance metric on contact.	15,549 players iPoker
6	Money coverage is a venue property	Winnings are recorded per hand for every seat or none — never partially. Whether they reconcile against the pot is decided almost entirely by the venue.	6 venues 0%–97%

The one that shaped the whole money analysis

VENUE	HANDS	RAKE RECONCILES	...AND IS PLAUSIBLE	COVERAGE	\$ DROPPED BY THE SCREEN
PokerStars PS	8,293,797	2,231,818	2,231,818	26.9%	—
iPoker IPN	5,996,194	0	0	0.0%	—
PartyPoker PTY	3,092,685	437,142	341,781	11.1%	\$677,131
Ongame ONG	1,631,839	1,584,889	1,579,938	96.8%	\$303,037
Full Tilt FTP	1,283,795	344,262	344,059	26.8%	\$6,713
Absolute ABS	1,257,978	780,763	573,513	45.6%	\$267,789

“Reconciles” = the derived rake balances against the pot within the parser’s bound. “Plausible” additionally requires it to be at most 6% of the pot — the screen justified in §5.7. iPoker reconciles on nothing and is excluded from every money figure in this report. **Hand counts here are post-deduplication** and so run 547 lower on iPoker than the Silver counts in §3.2 — that is exactly the 147 reused ids of §5.5b, dropped before any join.

CONSEQUENCES WE HAD TO ACCEPT, NOT ENGINEER AROUND

iPoker reconciles on none of its 5,996,741 hands, so it is excluded from every money question in this report — that is 14.6% of players, stated every time a dollar figure appears. Behavioural features work on 100% of hands; money features do not. And the all-zero winnings arrays are **never imputed**: filling them would invent a break-even player who never existed.

3.5 Value — what the data is worth once it is engineered

The fifth V is the one most projects answer with a sentence about “actionable insight”. Here it has a figure, and the figure is the whole deliverable:

WEEKLY RAKE IN VIEW

\$2,114,626

across 77,268 players on five venues, in the last seven days of each venue's own window

AT RISK NEXT WEEK

\$604,163

28.6% of it — the sum of calibrated probability x weekly rake, player by player

RECOVERABLE AT A 10% BUDGET

\$421,064

the part a retention team with 7,727 calls a week can actually put a hand on

Priced across each venue's whole observation window this comes to **\$8,217,411** of rake on 15.56 M hands — \$0.53 a hand. Value here is not a proxy for engagement or a score out of ten; it is the operator's revenue line, reconstructed from the pot up and attributed to the accounts that paid it.

3.6 What this data does **not** contain

An honest data-understanding section says what is missing, because every one of these absences closed off an analysis someone would otherwise expect to see.

NOT IN THE DATA	WHAT IT RULES OUT
A rake column	Rake is derived from pot – paid-out winnings, never read. The entire apparatus of §5.7 exists because of this one absence.
Most hole cards	Only 22.4% of hands reach a showdown, so 77.6% of hole cards are never revealed . That is structural to poker, not a collection failure — but it means no model here can evaluate decision <i>quality</i> , only decision <i>frequency</i> .
Any bot label	There is no ground truth for automated play anywhere in the archive. This is exactly why the supervised question is lapse, which the data labels itself, and why we make no bot-detection claim .
Deposits, withdrawals, bonuses	Only money that crossed a table is visible. Lifetime value, promotional response and payment friction — the three things a retention team would normally join to — are all absent.
Identity across venues	Player codes are per venue. The same human on two sites is two rows and must never be merged ; every join key in this project is site + player_id.
More than 26 days	No seasonality, no true churn, no temporal backtest across months. It is why the label is short-horizon lapse and why §6.7 refuses to claim temporal generalisation.

VERIFIED — THE WINDOW IS 26 DAYS, NOT THE 23 THE FOLDER NAMES CLAIM

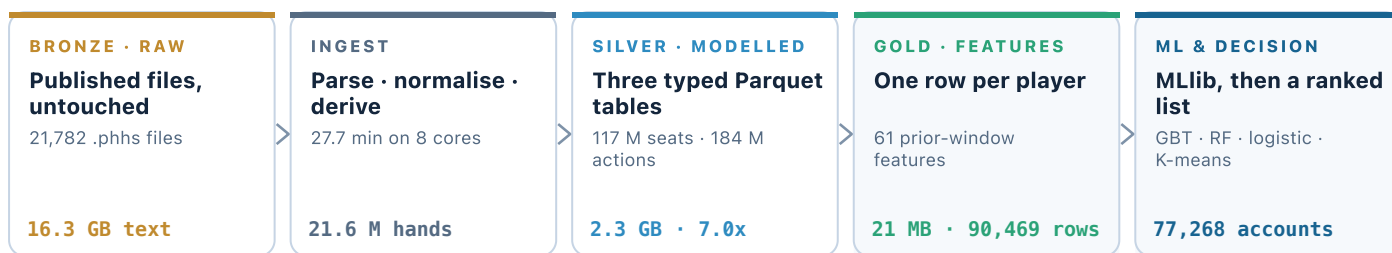
Every source folder is labelled 2009-07-01_2009-07-23, but the hands' own timestamps run to **day 26: 470,790 hands (2.2%)** fall on days 24–26, and they are structurally normal — same median pot, same stakes — from only 97 tables. It is a collection overrun on three venues, not corrupt dates. **The six venues therefore do not share a calendar**, and their last recorded days differ: ABS 20 · PS 20 · FTP 23 · ONG 24 · IPN 26 · PTY 26. This single fact decides the label definition in §6.2 — a global “played in the last week?” rule would mark **every Absolute and PokerStars player as lapsed by construction**, which is 44% of the dataset mislabelled before any model runs.

Provenance of this section. Every count, size and share on these three pages is recomputed from data/bronze, data/silver and data/gold by src/pipeline_facts.py, which writes docs/pipeline-facts.json and aborts if the reconciliation identity fails. The two figures it cannot recompute — the counts of hands *dropped* at parse time, which leave no row behind — are carried from the Silver build's own run log and labelled as such in that file.

4 Enterprise Architecture

ONE DIAGRAM, AND THE REASON EVERY LAYER EXISTS

THE PIPELINE, WITH WHAT WAS MEASURED AT EACH STAGE



116,621,636 seat-rows enter feature engineering **51x smaller** 21 MB is what every model actually reads

Every figure on this diagram is measured, not estimated — recomputed from the lake on disk by `src/pipeline_facts.py`. Local Spark (`local[8]`, 9 GB driver) on one laptop; the same code re-hosts to Databricks unchanged.

4.1 Why a medallion lake and not one big table

The three-layer pattern — **Bronze** raw, **Silver** modelled, **Gold** aggregated — is not decoration. Each boundary bought something specific, and we can name what.

LAYER	WHAT LIVES THERE	WHAT THE BOUNDARY BOUGHT US
Bronze	.phhs files copied byte-for-byte, partitioned <code>venue=.../stake=...</code>	Replayability. The parser was rewritten four times — and each rebuild re-read Bronze rather than re-downloading 16 GB. Folder names also carry free metadata (venue, date range, stake, big blind) that the files themselves do not repeat.
Silver	Three typed Parquet tables: <code>hands</code> , <code>hand_players</code> , <code>actions</code>	Parse once, query many. Parsing 21,782 TOML files takes 27.7 minutes; reading it back from Parquet takes seconds, and a query touching 2 of 20 columns reads 2 columns. It is also where the 7.0x compression lives.
Gold	One row per player — 90,469 rows, 61 columns	The models never touch the big data. Every experiment after feature engineering — the bake-off, three classifiers, two clustering algorithms, the calibration refit, four sensitivity re-runs — reads a 21 MB file. That is why iteration was cheap enough to find the leak in §5.6.

THE SENTENCE THIS ARCHITECTURE EXISTS TO EARN

116,621,636 seat-rows in. 21 MB out. The distributed work happens once, and the answer fits in memory.

4.2 Where the rubric's architecture spine actually sits in the repository

SPINE STAGE	IMPLEMENTATION	EVIDENCE IN THIS REPOSITORY
Business problem	—	§2 of this report; <code>docs/poker-project-plan.html</code>
Data sources	Zenodo / GitHub sparse checkout	<code>data/README.md</code> documents both routes
Ingestion	Python 3.13 + <code>tomllib</code> , 8-way parallel	<code>src/parse_phh.py</code> · <code>src/build_silver.py</code>
Distributed storage	Partitioned Parquet lake (HDFS-equivalent semantics)	<code>data/bronze silver gold</code> , 44 parts per Silver table
Data engineering	PySpark 4.2, <code>local[8]</code>	<code>src/features.py</code> · <code>src/features_lapse.py</code>
Feature engineering	PySpark aggregation to player grain	§5.6; 61 columns in <code>gold/player_lapse</code>
Machine learning	MLlib — GBT, RF, logistic, K-means, bisecting	<code>src/fork_a.py</code> · <code>src/segments.py</code>
Business insights	Money attribution + calibration	<code>src/rake_at_risk.py</code> ; §7
Executive recommendations	Ranked call list + dashboard + deck	§8; <code>docs/dashboard.html</code> ; the 10-slide deck

4.3 Phase 1 — acquisition and storage

Two download routes exist and they are not equivalent. The Zenodo archive is the complete, citable record — **exactly 20,289,230,983 bytes** — and is the right thing to reference in a paper. For building, we used the GitHub route with a **blob-less sparse checkout**:

```
# clone the tree without any file contents, then fetch only the folder we need
git clone --filter=blob:none --sparse https://github.com/uoftcprg/phh-dataset
git sparse-checkout set data/handhq # -> 21,782 .phhs files, 16.3 GB
```

This matters as an engineering decision, not a convenience: `--filter=blob:none` makes the initial clone almost instant and then streams blobs on demand, so a wrong path costs seconds rather than a 20 GB download. It also caused the one measurement error worth recording — the size was read *before* the sparse checkout finished materialising blobs and logged as 1.71 GB, which is why every figure in this report is now taken from a completed run rather than a running one.

Partitioning, and why it is the storage decision that matters

Bronze is laid out `venue={SITE}/stake={BB}/` and Silver is written in 44 parts per table. Both choices are about **partition pruning** — a query for one venue reads one venue’s files and skips the rest — and both mirror what HDFS or S3 would do behind Databricks. The layout also survives the thing that actually happens in projects: three separate rebuilds of Silver, each of which re-read Bronze without re-acquiring it.

STORAGE ECONOMICS, MEASURED

Bronze **16.3 GB** of text → Silver **2.3 GB** of Parquet, a **7.0×** reduction, while adding types, column statistics and predicate push-down. On cloud storage at list prices that is the difference between paying for 16 GB scanned per query and paying for the two columns a query actually reads. **Compression is a cost line, not a tidiness preference.**

4.4 Compute — and the four settings that made it work on a laptop

Everything in this report was produced on one **Apple M4, 10 cores, 16 GB RAM**. Spark runs in local mode, which is the same execution engine a cluster runs, with the driver and executors in one JVM. Four settings were not defaults and each fixed a specific failure:

SETTING	WHY IT IS NOT THE DEFAULT VALUE
<code>spark.driver.memory = 9g</code>	Default is 1 GB , measured. A 116.6 M-row shuffle does not fit in it. (Widely repeated advice says this cannot be set from the builder in local mode; we measured that it can — JVM max heap goes 1.0 GB → 9.0 GB.)
<code>spark.sql.shuffle.partitions = 96</code>	The default 200 is tuned for a cluster; 48 was too coarse and spilled. 96 is the value that holds a 116.6 M-row shuffle inside a 9 GB heap.
<code>spark.local.dir = data/_work/...</code>	Never /tmp. Spark spills many GB of shuffle data there and macOS tidies /tmp underneath long jobs. It killed two full runs ten minutes in, with a <code>FileNotFoundException</code> at a shuffle <i>read</i> that reads like a Spark bug and is not one.
<code>PYSPARK_PYTHON = sys.executable</code>	Without it Spark launches Python workers under whatever python3 wins the PATH — on macOS, Apple’s 3.9, which cannot import PySpark 4.2 at all. The failure surfaces as a <code>TypeError</code> buried in a Java stack trace that never mentions Python.

The rule that keeps local Spark alive: never `.collect()` or `.toPandas()` anything above Gold grain. On a laptop the driver *is* your 16 GB, and this is the single most common way to kill a local job. Every script in `src/` collects only at player grain.

4.5 What happens at ten times the scale

The honest test of an architecture is what breaks when the data grows. Three of the five stages do not change at all, one changes linearly, and exactly one needs a cluster.

STAGE	TODAY	AT 10× (OR A LIVE OPERATOR'S FEED)
Acquisition	16.3 GB, once	Changes shape. A batch archive becomes a stream; Kafka topics per venue land straight into Bronze. Cost is storage, which is linear and cheap.
Parse → Silver	27.7 min, 8 cores	Scales linearly and embarrassingly. The unit of work is one file; nothing is shared. Ten times the files on ten times the cores is the same wall clock.
Feature engineering	4.6 min, Spark	The only stage that genuinely needs a cluster. It is a shuffle over every seat-row, and shuffles are where memory pressure lives. This is the stage Databricks exists for.
Model training	5.0 min	Does not change. Gold grows with <i>players</i> , not with hands. Ten times the hands from the same population is the same 21 MB table.
Serving the list	0.2 s, no Spark	Does not change. The dashboard payload is 77,268 rows; it would still open instantly at a million.

THE SCALABILITY CLAIM, STATED PRECISELY

Volume scales the **storage and one shuffle**. It does not scale the modelling, the evaluation or the decision surface, because feature engineering is a one-way funnel from **116,621,636** rows to **90,469**. Getting that boundary in the right place is the architecture decision; everything else is configuration.

4.6 The Databricks path, and what we did not use

Phase 2 of the brief specifies Databricks and PySpark. The engineering here is written as PySpark running against a partitioned Parquet lake, which is what a Databricks workspace runs on top of, so moving it is **a re-hosting job, not a rewrite**: the DataFrame code is unchanged, the paths become dbfs:/ or a mounted bucket, the Silver and Gold tables become Delta tables to gain transactions and time travel, and `local[8]` becomes a cluster spec. Nothing in `src/features_lapse.py`, `src/fork_a.py` or `src/segments.py` depends on running locally.

Note: Databricks Community Edition was retired 1 Jan 2026; the free tier is now Databricks Free Edition.

Deliberate omissions, with reasons

TECHNOLOGY	USED?	REASONING
HDFS	equivalent	A partitioned Parquet lake on local disk gives the same semantics the pipeline depends on — immutable blocks, partition pruning, parallel readers. Standing up a single-node HDFS would have added an operational layer without changing a single line of the analysis.
Kafka	no	Optional in the brief. With a fixed historical archive there is no stream to consume; building one would have been theatre. It is the right answer for the production version and is named in the roadmap.
HBase	no	Optional. The access pattern is analytical scans over columns, not random point lookups by key. Parquet is the correct store for that; a wide-column store would be the wrong tool chosen for the marks.
Sqoop / Flume	no	Both move data out of relational databases or log tails. The source here is a published file archive, so the ingestion mechanism is a filtered clone, documented in §4.3.
Polars	yes	Used deliberately <i>alongside</i> Spark, not instead of it: for the per-file parse (where the unit of work fits in memory and Spark's overhead is pure cost) and for the dashboard and deck payloads, which run in 0.2 s and 2 s respectively. Choosing the smaller engine where it wins is an architecture decision too.

WHERE THIS ARCHITECTURE IS WEAKEST

It is a **batch lake with no orchestration layer** — no Airflow, no job scheduler, no automated freshness check. Re-running the chain today means running six scripts in a documented order. That is defensible for an eight-week engagement producing one decision list, and it would not survive contact with a weekly production cadence. It is the first item under “engineering” on the roadmap in §8.3.

5.1 Parsing — reading a game, not a table

The raw layer is not rows. Each hand is a TOML table describing a small event log: who sat where, what the blinds were, and an ordered list of actions written in a compact notation. Turning that into three flat tables is where the domain lives.

```
# one hand, abbreviated -- .phhs files are valid TOML, so no regex parser is needed
[[hand]]
  variant = "NT"          # no-limit Texas hold 'em
  players = ["p1", "p2", "p3", "p4", "p5", "p6"]
  blinds_or_straddles = [0.10, 0.25, 0, 0, 0, 0] # p1 = small blind, p2 = big blind
  starting_stacks = [25, 31.4, 25.0, 18, 25, 40.5] # int AND float, in one list -- defect 1
  actions = ["p3 f", "p4 cbr 0.75", "p5 f", "p6 cc", "p1 f", "p2 cc", ...]
```

Two pieces of notation carry the whole behavioural feature set, and getting either wrong quietly poisons every model downstream:

- **cc means check or call** — the same token for “I put in nothing” and “I matched a bet”. It is disambiguated by comparing the current maximum street bet against that player’s own street contribution.
- **cbr means bet or raise, and its number is the cumulative street total**, not an increment. Treating it as an increment inflates every aggression figure.

THE CORRECTNESS TEST THAT GUARDS THIS — A BUSINESS NUMBER, NOT A UNIT TEST

The big blind has already paid, so pre-flop it can **check for free**, and that check is written with the same cc token as a call. Count it as voluntary and every player’s VPIP inflates, and the segmentation becomes meaningless. The test is that **big-blind VPIP must land near the pooled rate, never near 100%**. Measured over all **21,556,835** big-blind seats at full scale: **31.3%**, against a pooled VPIP of **27.4%** and PFR of **14.1%**. The parser prints this on every run.

5.2 The reconciliation — every hand parsed, or dropped with a reason

Silent data loss is the failure mode a “justify every decision” brief is really testing, so the pipeline counts what it discards and the arithmetic must close exactly:

21,556,435	+	49,119	+	133	=	21,605,687	=	21,605,687
distinct hands in Silver		dropped — no big blind, so money cannot be normalised		dropped — no hand ID, so seats cannot be joined		accounted for		the dataset’s published count

The identity closes exactly — not to within a rounding tolerance, exactly. Silver holds 21,556,835 rows against 21,556,435 distinct keys; the 400-row difference is one venue reusing hand IDs (§3.4), not further loss. 0 of 21,782 files were skipped.

The two drop reasons are both principled. **No big blind** means money cannot be normalised, so the hand cannot be compared to any other; **no hand ID** means it cannot be joined to its own seats. Neither is imputable. Total loss is **0.23%**, and the remaining **400**-row gap between rows and distinct keys is the reused-ID defect from §3.4, not further loss.

5.3 The bug this pipeline exists to have caught

Rake is derived as $\text{pot} - \text{sum}(\text{winnings})$, and the pot has to account for the **uncalled bet**: if one player bets 40 and every-one folds, only the second-largest contribution was ever contested. The first version subtracted that uncalled amount from the pot — so rake reconciled perfectly — but left it inside the bettor’s own invested figure, so the player never got their own money back.

HOW WRONG IT WAS, AND THE IDENTITY THAT PROVED IT

Player profit was wrong on **99.4% of hands**, by a mean of **7.35 bb** — seventeen times the rake itself. Every high-volume player came out at roughly **−100 bb/100**, i.e. losing a full big blind per hand, which is impossible in a rake-only game. The test that caught it: **poker is zero-sum apart from the rake**, so summed over every reconciled hand, $\sum \text{net_bb}$ must equal $-\sum \text{rake_bb}$. Before the fix the identity failed by exactly the uncalled bet, on 100% of 852,293 hands — algebraic proof rather than a guess. After it, measured across the whole lake:

$\sum \text{rake_bb} = 2,064,329$ · $\sum \text{net_bb} = -2,064,330$ · ratio **1.000000**

This forced a complete rebuild of Silver, and it is worth saying why the check worked: **it is a statement about the business, not about the code**. “The operator’s take must equal the players’ combined loss” is something a domain expert can verify without reading a line of Python — and it caught a defect no schema validation or null check would have seen.

5.4 Missing values — one policy, applied four times

The brief asks how missing values were handled. Our policy is a single rule, and it is the opposite of the textbook default: **where a venue did not record something, the column is set to null and a boolean records that it was never observed. Nothing is imputed.** The reason is that in this dataset every plausible fill value is also a plausible *real* value, so an imputed number is indistinguishable from a measurement — and it lies in a specific, predictable direction.

WHAT IS MISSING	SCALE	WHAT IMPUTING IT WOULD HAVE DONE	WHAT WE DID
Table identity on iPoker	6.0 M hands 15,549 players	The obvious fill is <code>max_tables = 1</code> , a missing value wearing a plausible number . It would file every iPoker professional as a single-table recreational player — the opposite of the truth, in the one feature §6.3 shows can separate a professional from a customer.	Null + <code>tables_recorded = false</code> ; venue excluded from the ecosystem segmentation.
Starting stacks on iPoker	all hands	The file literally contains <code>inf</code> , a legal TOML float. It parses silently, survives to Gold, then detonates any scaler or distance metric on contact.	Guarded to null plus <code>stacks_recorded</code> . (The guard exists in the lapse feature build; back-porting it into the original Gold build is listed as outstanding in Appendix E.)
Winnings (all-zero arrays)	52.2% of hands	Filling zeros with a break-even outcome invents a player who never existed, and it would move the mean profit of an entire venue toward zero.	Filter, do not fill. The money filter is <code>rake_bb IS NOT NULL</code> , never “winnings present”.
Behavioural rates for thin players	varies	A VPIP computed from four hands is noise presented as a rate.	MLlib Imputer (median) + a binary missing-indicator column per imputed feature , so the model can learn from the fact of absence as well as from the fill.

Engineering note. MLlib's Imputer accepts only `DoubleType/FloatType` — an integer column fails the schema check rather than being promoted — so every numeric feature is cast to double up front.

5.5 Outliers — diagnosed, not clipped

The one column with a genuinely wild tail is `bb_per_100`, a player's profit rate. The median is a sane **−5.3** (roughly the rake drag you would expect), but **6.4% of players sit beyond ±200**, and the worst case is a player with 103 hands played, **7 of them with usable money**, at **−1,494 bb/100**.

THE DIAGNOSIS, AND WHY CLIPPING WOULD HAVE BEEN THE WRONG FIX

This is not a fat-tailed distribution of talent. It is a **sample-size artifact with a specific cause**: the population filter counts `hands_played`, but a profit rate can only be computed on hands where money reconciles — and the median player has only **92** such hands. Winsorising or clipping would have hidden the cause and kept the bias. Instead Gold stores the raw value *plus both counts*, and any model touching money filters on **`money_hands`, never `hands_played`**. The threshold was chosen by sweeping it rather than by convention: **≥1** keeps 88,655 players with 6.4% extreme, **≥30** keeps 81,853 at 4.4%, **≥50 keeps 65,732 at 2.8%**, **≥100** keeps 41,654 at 1.6%. Fifty is the knee — it more than halves the artifact while keeping three quarters of the money-bearing players. **A judgement call, recorded as one.**

Genuine outliers that are *not* errors were kept. The largest pots are real hands and the players with 8,799 prior-window hands are real professionals; removing them would delete the Grinder segment, which is the finding.

5.5b De-duplication before any join

Because one venue reuses hand IDs (§3.4, defect 4), the join key `hand_uid = site:hand_id` is correct but **not unique** — 147 ids across 547 rows. **Both** rows are dropped rather than one kept arbitrarily: once two different hands share an id, a seat-row cannot be attributed to the right one. The cost is 0.0025% of hands; the alternative is a wrong number nobody would have noticed.

5.6 Feature Engineering

TURNING 116.6 M SEAT-ROWS INTO 61 COLUMNS A MODEL CAN LEARN FROM

The design constraint that produced every feature

The model predicts what a player does in the seven days *after* a cutoff. Therefore every feature must be computable on the cutoff day, from hands played on or before it. That is not a modelling nicety — it is the difference between a feature set and a leak, and it is enforced structurally: all thirty modelling columns are recomputed with a `day <= cutoff` filter and prefixed `p_`, rather than re-filtered from an existing lifetime aggregate.

```
# the cutoff is per venue, because the six venues do not share a calendar (see 3.6)
cutoff = site_last_day - 7 # ABS/PS: day 13 · FTP: 16 · ONG: 17 · IPN/PTY: 19
prior = seats.filter(day <= cutoff) # 93,430,449 seat-rows -> every p_ feature
label = seats.filter(day > cutoff) # 23,188,818 seat-rows -> the answer only
population = prior.groupBy(site, player_id).count() >= 100 # PRIOR hands, not lifetime
```

The four families, and what each one is for

FAMILY	COLUMNS	WHAT BUSINESS QUESTION IT ENCODES
Recency & rhythm	p_recency_days, p_active_day_share, p_active_minutes, p_hands_per_minute, p_tenure_days	"Are they still showing up, and how much of their available time do they spend at a table?" The strongest family by a wide margin — and we say so rather than overselling the behavioural ones.
Decay	p_hands_w1, p_hands_w2, p_hands_last3, p_late_share, p_last3_share	"Is their volume ramping into the cutoff or falling away from it?" A player at 500 hands a week is a different risk depending on whether last week was 200 or 900. These are new columns, not re-filtered ones — the original Gold table could not express a trend at all.
Style	p_vpip, p_pfr, p_vpip_pfr_gap, p_aggression_factor, p_wtsd, p_fold_rate, p_avg_invested_bb, p_bb_vpip_rate	"What kind of player are they?" This is the family that makes the segmentation possible and that connects the churn model to the ecosystem thesis.
Context	p_max_stake, p_min_stake, p_distinct_stakes, p_seat_count, p_btn_share, p_prior_span_days, site dummies	"Where do they play, and how comparable is their record to anyone else's?" <code>p_prior_span_days</code> exists so the model can calibrate for the fact that some venues offer a 13-day prior window and others 19.

Two engineered features worth naming individually

- **Multi-tabling** (`p_max_tables`, `p_avg_tables`, `p_distinct_tables`) — distinct table identities a player occupied in the same minute. One human plays one table; a professional plays a dozen. §6.3 shows this is **the only feature that separates a professional from a customer at all**: on betting style alone, no professional cluster forms.
- **Position** (`p_btn_share`) — free, and free features are the best kind. Players are indexed in blind order, so `p1` is the small blind, `p2` the big blind, and the *highest* index is the button. It holds for three or more players; heads-up differs and is excluded rather than approximated.

Encoding and scaling

TRANSFORM	APPLIED TO	WHY THIS AND NOT THE ALTERNATIVE
Big-blind normalisation	every money column, at parse time	\$50 at 25NL and at 1000NL are not the same event. 200 bb vs 5 bb is. Done in Silver so no downstream stage can forget.
One-hot, reference level	site → 5 dummies	PokerStars, the largest venue, is the dropped reference. Six venues with wildly different prevalence must be visible to the model, not absorbed into the intercept.
Median imputation + indicator	8 columns with venue-driven nulls	The indicator is the point: "this venue never recorded it" is itself predictive , and a bare median throws that signal away.
Standard scaling	logistic regression and K-means only	Distance-based and coefficient-based methods need it; tree ensembles are invariant to monotone rescaling and do not. Applying it to the trees anyway would be cargo cult.
Log1p	the volume target in the rejected value model	Hand counts span four orders of magnitude. (Recorded here because the model was built and rejected — see §6.1.)

VERIFIED — TWO INDEPENDENT CHECKS THAT THE FEATURE TABLE IS CORRECT

1. Arithmetic closure. `p_hands + hands_final7 == hands_played`, checked row by row against the separately-built lifetime table on all **90,469** players — **zero mismatches**. 2. Behavioural sanity. Prior-window pooled VPIP **27.2%** / PFR **13.7%** / big-blind VPIP **30.7%**, against the parser's lifetime 27.4% / 14.1% / 31.3% — close, and nowhere near the ~100% that would mean the check-versus-call bug had returned.

The two leaks we found, and how each was proved

Both are worth reading as method rather than as confession: neither was caught by a metric looking wrong. One was caught by arithmetic and one by algebra, and a third — the one everybody warns about — was caught by an ablation that came out backwards.

Leak 1 · the population filter was built out of the answer

The obvious way to define a modelling population is "players with at least 100 hands". But that counts hands over a player's *whole history*, including the week being predicted. A player with 30 prior hands can only clear 100 by playing 70 more *in the label week* — that is, by definitely not lapsing. The filter would then contain the label.

THE ARITHMETIC IS DECISIVE, NOT SUGGESTIVE

POPULATION RULE	PLAYERS	LAPSED	PREVALENCE
Lifetime "≥ 100 hands" rule counts the week being predicted	98,718	61,702	62.5%
Prior-window "≥ 100 hands" rule applicable on the cutoff day	90,469	61,702	68.2%
Difference	8,249	0	—

The lifetime rule admits **8,249 extra players** and every single one of them is labelled "active" — not approximately, exactly, because the lapsed count is identical in both populations. The entire difference is non-lapsers admitted by a filter that had already seen the answer. Selecting on **prior-window hands** instead is a rule that could actually be applied on the cutoff day. Consequence for reading the metrics: **prevalence moves from 62.5% to 68.2%**, so PR-AUC in §6 is not comparable to the earlier prototype's, and ROC-AUC — which is prevalence-independent — is the number we quote across both.

Leak 2 · a ratio that reconstructs the label algebraically

A feature called `hands_per_day` looks innocuous. It is `hands_played / days_active`, and `hands_played` is `hands_prior + hands_final7`. With `hands_prior` also in the feature set, a model can recover the target outright:

```
hands_final7 = hands_per_day × days_active - hands_prior    # exact, not approximate
```

The first prototype scored 0.960 PR-AUC on precisely that. The lesson generalises past this column: **a ratio whose denominator or numerator spans the label window is a leak even when neither component individually is**. The same reasoning excluded `money_coverage`, which shares the denominator.

Leak 3 · the one everybody warns about, tested rather than assumed

Every rate column in the original Gold table — VPIP, PFR, fold rate, showdown rate, aggression — averages over a player’s entire recorded history, *final week included*. A prototype built on those columns is reading its own answer. The test is an ablation with the same estimator, population and split, across three feature sets:

FEATURE SET	FEATURES	ROC-AUC	PR-AUC
The old floor — volume and tenure only honestly prior-window, but only 2 features	2	0.690	0.805
Lifetime rates only — the leaky set every column averages over the label week	11	0.819	0.911
Prior-window only — this build structurally cannot see the week being predicted	30	0.856	0.917

Same estimator, same population, same split — only the feature set changes.

THE RESULT THAT JUSTIFIED REBUILDING THE FEATURE TABLE

In the prototype the ordering was the other way round: the leaky lifetime rates *beat* the honestly-prior features (0.886 vs 0.798 PR-AUC), which is the signature of leakage rather than of behaviour. After the rebuild it has reversed — **the honest feature set out-predicts the leaky one**, and it got *better*, not worse, in the process. Two cautions so this is not oversold: the honest claim rests on **construction** (`day <= cutoff`), not on this table, and the two rows are not a controlled comparison of equal capacity (30 features against 11). What the ablation rules out is the specific failure the prototype found — a model that *needs* the leaky columns to perform.

Two feature sets, and why the bigger one lost

Three of the columns a retention analyst would most want — stack depth, multi-tabling and realised money — do not exist on iPoker. That forces a choice, so we built both and measured it.

FEATURE SET	FEATURES	COVERAGE	GBT ROC-AUC	VERDICT
CORE	30 + 2 indicators + 5 venue dummies	6 venues 90,469 players	0.857	The spine. Keeps every venue, so the deliverable is a list for the whole business.
EXTENDED	CORE + stacks + tables + money	5 venues 77,268 players	0.863	Buys +0.006 ROC-AUC for the loss of 14.6% of players. Not worth excluding a whole venue for. Reported rather than imputed — a median stack on a venue that recorded none is a number nobody measured.

5.7 Reconstructing the revenue line

THE OPERATOR'S OWN INCOME IS NOT A COLUMN IN ITS OWN DATA

Nothing in this dataset records what the house took. Rake has to be derived from what the players put in and what they got back:

```
pot = sum(contributions) - (highest - second-highest contribution) # the uncalled bet, returned
rake = pot - sum(winnings) # accepted only if <= 15% of the pot
```

That works when a venue's export is complete. When it is not, the shortfall lands in the residual and is indistinguishable from rake. And the acceptance test itself creates a bias, because a fixed shortfall is 40% of a small pot and 3% of a large one — **so the hands that survive are the big ones**. Measured, this is not a subtle effect:

WHERE RAKE RECONCILES				WHERE IT DOES NOT		
VENUE	HANDS	MEDIAN POT	SAW FLOP	HANDS	MEDIAN POT	SAW FLOP
PokerStars	2,231,818	2.50	49.6%	6,061,979	3.00	54.8%
Ongame	1,579,938	2.50	44.4%	51,901	56.50	87.4%
Absolute	573,513	3.00	50.7%	684,465	6.50	63.7%
Full Tilt	344,059	2.50	46.3%	939,736	2.50	46.2%
PartyPoker	341,781	23.40	99.5%	2,750,904	2.50	39.0%

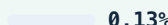
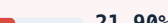
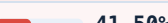
Pots in big blinds. The hands whose money reconciles are systematically the bigger ones, because a fixed shortfall is a large share of a small pot and a small share of a large one. On PartyPoker the two populations differ by a factor of nine.

WHY THE OBVIOUS METHOD IS WRONG, AND BY HOW MUCH

The natural shortcut is: take each player's observed rake per 100 hands and scale it to all their hands. On PartyPoker the hands where rake reconciles have a **median pot of 23.4 bb** and **99.5% saw a flop**; the rest have **2.5 bb** and **39.0%**. Scaling off the first group charges every hand at the rate of the biggest pots and ignores that most hands never see a flop and pay nothing at all. We computed it anyway, as a sensitivity: it runs **25% high** — \$2,483,876 against \$1,981,801 on the same players — and high is the direction that flatters the answer.

The plausibility screen — a measurement, not a threshold we liked

The parser's 15% acceptance bound was deliberately loose, chosen when there was nothing better to test against. There is now something better. **PokerStars' exports reconcile completely, and not one of its 1,107,219 flop hands rakes above 5.0% of the pot**. Full Tilt and Ongame agree. Two venues do not, and the disagreement is enormous:

VENUE	FLOP HANDS	MEDIAN RAKE SHARE	90TH PCT ABOVE THE 6% CEILING	AT THE PARSER'S 15% BOUND
PokerStars	1,107,219	4.26%	5.00% 	0.00%
Full Tilt	159,399	4.55%	5.00% 	0.01%
Ongame	705,887	4.89%	5.00% 	0.07%
PartyPoker	435,472	4.94%	13.33% 	7.98%
Absolute	497,285	5.00%	11.33% 	3.13%

Rake as a share of the pot, on hands that saw a flop. The median is the real commercial rate on every venue — strong evidence the derivation is sound. The upper tail is not: no operator charges 15%.

No commercial operator has ever charged 15%. That upper tail is **missing money wearing a rake's clothes**, it sits precisely on the players who contest the biggest pots, and it is worth \$267,789 on Absolute and \$677,131 on PartyPoker. So a hand counts as **measured** only if its derived rake is at most **6%** of the pot — a 20% margin above the highest value the clean venues ever produce. Everything else is treated exactly like a hand with no figure at all, and priced by the estimator below.

WHY THIS IS THE RIGHT KIND OF THRESHOLD

It is not tuned to an outcome. It is **calibrated against a venue whose data is known to be complete**, and the script re-prints the justifying table on every run — so if a future Silver rebuild changes the picture, the run says so instead of the constant quietly going stale. That is the difference between a defensible cut-off and a convenient one, and it is the kind of decision the "justify each engineering choice" requirement is asking for.

The estimator — charge every hand what hands like it actually paid

Conditional-mean imputation over four columns that are recorded for **100% of hands**: the venue, the big blind, whether a flop was dealt, and the pot size bucketed in big blinds.

```
rake_est_bb = mean(observed rake) over (site, big_blind, saw_flop, pot_bucket)
  → fall back to (site, saw_flop, pot_bucket)      # same venue, other stakes
  → fall back to (saw_flop, pot_bucket)            # last resort
rake_est_usd = rake_est_bb × big_blind              # dollars only at the very end
```

Three decisions inside that, each of which was a bug first:

- **Fit in big blinds, never in dollars.** A dollar cell cannot be borrowed across stakes. The first version priced 59,910 PokerStars 25NL hands at **\$29.79 each**, from a fallback cell built mostly out of Ogame hands at a \$10 big blind. In big blinds the same fallback is approximate instead of absurd.
- **Bucket finely at the bottom, bounded at the top** — every whole big blind to 20, then 20/30/50/100/200/500/1000. Where the rake cap bites, the rake share falls from 5% to under 1% between a 20 bb pot and a 200 bb one; one bucket spanning both would average two different rake structures together.
- **Where rake is measured, the measurement is used.** Nothing is modelled over the top of it.

Validated out of sample — this is the estimator's error bar

Cells were fitted on half the measured hands (split on a hash of the hand key) and scored against the other half:

VENUE	HELD-OUT HANDS	ACTUAL	PREDICTED	RATIO	MAE / HAND	MEAN / HAND
PokerStars	1,116,165	\$239,960	\$241,019	1.0044	\$0.0172	\$0.2150
Ogame	790,027	\$727,202	\$731,109	1.0054	\$0.1710	\$0.9205
Absolute	286,797	\$111,401	\$111,733	1.0030	\$0.1048	\$0.3884
Full Tilt	171,955	\$107,357	\$108,009	1.0061	\$0.0385	\$0.6243
PartyPoker	171,007	\$281,266	\$279,283	0.9929	\$0.7570	\$1.6448

Cells fitted on half the measured hands, scored against the other half. **Every venue lands within 0.7% on totals**; per hand it is much rougher, which is why this estimator is quoted for sums and never for a single hand.

HOW TO QUOTE THIS HONESTLY

Every venue lands within 0.7% on totals. Per *hand* it is much rougher — PartyPoker's mean absolute error is **\$0.76** against a mean of **\$1.64**. So: this estimator is trustworthy for sums over thousands of hands and **should never be quoted for one player's single hand**. Of the dollars that had to be estimated, **87.1% came from the most specific cell** (same venue, stake, flop status and pot bucket), 1.7% from the second level and 11.1% from the last resort. By hand count the last resort looks larger (27.8%), but 90% of those hands are pre-flop folds worth exactly \$0.00 — which is why the split is reported in dollars as well as in hands.

Attribution — whose rake is it?

Rake comes out of the pot, not out of a player, so this is a modelling choice and not a fact. We use **contributed rake**, the industry-standard answer: a player is charged in proportion to what they put into the pot, so someone who folded pre-flop is charged nothing and the two players who built a 40 bb pot are charged in proportion. **86,216,438** seat-rows are attributed this way — the full seat join, less iPoker.

TWO REGRESSION TESTS THAT RUN ON EVERY EXECUTION, BOTH GREEN AT FULL SCALE

TEST	WHAT IT CATCHES	MEASURED
Contributed vs equal-split totals must be identical	Shares sum to 1 inside a hand, so both rules must return the same grand total. Any difference means the join fanned out on the non-unique hand key — a live risk, not a theoretical one.	\$8,217,411 both ways ratio 1.000000
Prior-window hand counts must equal the feature table, player by player	Catches both a fan-out and any drift in the per-venue cutoff between the money script and the feature build. The money and the label have to describe the same week or the multiplication in §6.4 is meaningless.	0 mismatches on all 77,268 players

What contributed rake does and does not mean. It measures **who paid** the rake. It does not measure **whose money funded it** — and in this business those are different populations. That distinction turns out to matter more than any other single point in this report, and it is taken up in §7.3.

6.1 Choosing the question before choosing the model

A dataset this rich supports several defensible questions, and picking one by intuition is how projects end up defending a model nobody needed. So we **prototyped five candidate questions in scikit-learn first** — deliberately throwaway code, 41 seconds to run — scored each against its own stated dumb baseline, and used the result to choose what to build properly in MLLib.

CANDIDATE QUESTION	ITS OWN DUMB BASELINE	BEST PROTOTYPE	SCORE /20
Lapse classification all six venues; a retention list a team can work through on a Monday	majority class PR-AUC 0.625	PR-AUC 0.927, ROC 0.877	17
Player value · money cleanest methodology; loses on coverage — five venues, 61% of players	per-venue mean RMSE 82.64	RMSE 72.82, skill +0.224	15
K-means segmentation weak separation, but it ships regardless as the base layer	— —	silhouette 0.279 at k=2	13
Player value · volume highest raw skill and least trustworthy — target is zero for 62.5% of players	per-venue mean RMSE 2.464	RMSE 1.866, skill +0.427	13
Hand economics · pot size eliminated on evidence — a genuine negative result	per-stake mean RMSE 36.92	RMSE 36.84, skill +0.004	12

The score out of 20 is four axes of five: **signal** and **coverage** are measured from the run; **business story** and **rubric fit** are judgement calls and are marked as such. Two algorithms were fitted per supervised candidate (a linear model and a gradient-boosted tree), which also de-risked the “at least two approaches” requirement.

Skill = 1 – SSE(model)/SSE(baseline). Plain R² measures against the global mean, which flatters any model whose baseline is already per-group, so it is not used here.

Why each candidate placed where it did

- **Lapse classification — the winner.** Best combination of coverage (all six venues) and operator action: a retention list is something a team can literally work through on a Monday. Its honest floor — prior-window features only — still beat the baseline decisively, so the signal survives deleting every leakage-suspect column.
- **Player value in money — the close second, and methodologically the cleanest.** Real signal, positive on every venue independently, and it makes no forward-looking claim so lifetime aggregation is correct rather than leaky. It lost on **coverage**: iPoker reconciles no money at all and the money_hands ≥ 50 filter drops 38.6% of players. It also has a framing gap — bb_per_100 is **the player’s own profit, not the operator’s revenue from them** — which a probing examiner would find. Rather than shelve it, we answered the money question a different way in §6.4, with rake instead of profit.
- **Player value in volume.** Highest raw skill of any candidate (+0.427) and the least trustworthy: the leakage ablation hits it hardest, and the target is zero for 62.5% of players — a classification problem wearing a regression’s clothes.
- **Hand economics — eliminated on evidence, and worth an honest line.** Predicting a hand’s pot size from what is knowable before the deal scored a skill of **0.004**. Ridge and gradient boosting both land within 0.2% of a per-stake mean, on every stake separately. **The cleanest fork methodologically turned out to have nothing to predict** — pot size is driven by the cards dealt and the decisions players make, none of which is knowable in advance. Finding that in an afternoon is exactly what the bake-off was for.

THE DECISION, AND ITS CONDITION

Lapse classification as the spine, with K-means as the unsupervised base layer — *conditional on* rebuilding the feature table with prior-window-only columns first, because without them the headline metric was not defensible. That condition is what produced §5.6, and closing it reversed the leakage ablation. **The rejected candidates are not wasted:** the volume model is why we know recency dominates, the hand model is a documented negative result, and the value model is the second stage recommended in §8.3.

Note on comparing prototype metrics to final ones. The prototype ran on the lifetime-filtered population at 62.5% prevalence; the shipped models run at 68.2% after the selection leak in §5.6 was closed. **PR-AUC is not comparable across the two** because it moves with prevalence; ROC-AUC is, and is what we quote when comparing.

6.2

Model 1 · Supervised classification — who goes quiet next week

GRADIENT-BOOSTED TREES · RANDOM FOREST · LOGISTIC REGRESSION, IN PYSPARK MLLIB

The label, and what it is not

Lapsed = 1 if the player logged **zero hands in the seven days ending on their own venue’s last recorded day**. Measured per venue, never against a global calendar — §3.6 shows why a global window would mislabel 44% of the dataset by construction.

THIS IS SHORT-HORIZON LAPSE, NOT CHURN — AND WE SAY SO EVERY TIME

Twenty-six days of data cannot show that anyone left for good. A week of silence is an **early warning**, and the business action it justifies is a phone call or an offer, not writing the account off. Every use of the word “churn” in this report is either in quotation marks or is describing what we deliberately did not claim.

The population, and why per-venue reporting is mandatory

90,469 players with at least 100 prior-window hands, **67,873** train / **22,596** held out, split on a hash of the player key so it is identical on any machine. Prevalence varies enormously by venue, which is why every metric below is also reported per venue rather than pooled:

VENUE	HELD-OUT PLAYERS	LAPSE PREVALENCE	ROC-AUC	PR-AUC
PokerStars PS	12,187	77.8%	0.861	0.945
iPoker IPN	3,236	56.4%	0.800	0.807
PartyPoker PTY	3,186	73.3%	0.817	0.905
Absolute ABS	1,679	45.0%	0.780	0.719
Full Tilt FTP	1,339	41.7%	0.812	0.751
Ongame ONG	969	48.6%	0.769	0.725

Shipped GBT, scored on each venue’s own held-out players. **The model is weakest exactly where lapsing is rarest** — small test folds explain part of it, not all. This is the evidence behind recommendation 3.

Results — three algorithms against three baselines

APPROACH	ROC-AUC	PR-AUC	BEST F1	PRECISION @ 10% BUDGET
Baseline · flag every player no ranking at all	0.500	0.683	—	—
Baseline · sort by days since last seen four minutes in a spreadsheet	0.800	0.864	—	0.885
MLlib · logistic regression readable coefficients	0.837	0.901	0.857	0.944
MLlib · random forest 100 trees, a statistical tie with GBT	0.857	0.918	0.865	0.966
MLlib · gradient-boosted trees the shipped model	0.857	0.917	0.866	0.970

CORE feature set, all six venues, 22,596 held-out players. PR-AUC’s floor is the prevalence (0.683), not 0.5. **The second row is the bar every line of this pipeline has to clear.**

Gradient boosting and random forest are **indistinguishable** (0.8566 vs 0.8569 ROC-AUC — noise). Logistic regression gives up about 0.019 of ROC-AUC and returns readable coefficients for it. **That trade is the comparison the brief asks for**, and the gap is small enough that the linear model is a legitimate choice if a slide has to explain *why* a particular player was flagged.

The baselines, and the one that nearly won

A score with nothing beside it is not evidence. Three baselines were built before any model was trained, and one of them is genuinely hard to beat.

BASELINE 1 · FLAG EVERYONE
ROC 0.500

Recall 1.0 by construction, precision = prevalence, F1 0.811. This is what “accuracy” flatters, and it is why accuracy appears nowhere in this report.

BASELINE 2 · SORT BY DAYS SINCE LAST SEEN
ROC 0.800

No model, no training, no data engineering — a retention manager can do this in a spreadsheet in four minutes. **This is the bar MLLib has to clear to justify its place in the architecture.**

BASELINE 3 · “QUIET FOR A DAY” AS A RULE
F1 0.856

A deployable if-statement. At the default 0.5 threshold **every model loses to it** — GBT scores 0.829.

THE THRESHOLD IS A DECISION, NOT A DEFAULT — AND IT IS WORTH STATING PLAINLY

0.5 is an inherited convention from a balanced world. On a 68/32 problem it is an arbitrary choice that happened to make a working model look worse than an if-statement. Chosen deliberately, GBT reaches **0.866** at a threshold of **0.25** and wins. **We report the loss as well as the win**, because a model that only beats its baseline after an unstated tuning step is not actually beating it.

From a metric to a budget line

A retention team has capacity, not an appetite for “contact all 15,423”. The operating question is what a **fixed contact budget** buys, and that is where the model earns its place:

WHO GETS THE 2,260 CALLS	TRUE LAPSERS REACHED	PRECISION	LIFT VS RANDOM
Random selection	1,543	0.683	1.00x
Sort by recency — free, no model	2,001	0.885	1.30x
MLlib gradient-boosted trees	2,192	0.970	1.42x

A 10% contact budget on the 22,596 held-out players. **Recall at this budget is 0.142** and that is honest: when 68% of players go quiet in a week, no list covering a tenth of them can cover most of them.

Against the free baseline the model finds **191** more genuinely at-risk players per 2,260 contacts — equivalently, **191 fewer bonus offers posted to people who were never leaving**, every week. That is the sentence that turns a metrics table into a budget line.

THE HONEST READING OF RECALL AT A 10% BUDGET

Recall at that budget is only **0.142**, and that is not a defect to hide. When 68% of the population goes quiet in any given week, **no list covering 10% of players can cover most of them**. Which is precisely why the interesting question is not “who is lapsing” but **“which lapsers are worth a call”** — and answering that is what §6.4 exists for.

What the model is actually using

Interpretability was a selection criterion, not an afterthought. Both a tree ensemble’s importances and a linear model’s standardised coefficients point the same way:

GBT FEATURE IMPORTANCE			LOGISTIC REGRESSION • STANDARDISED COEFFICIENTS		
recency days		0.249	recency days		+0.642
hands last3		0.088	hands last3		-0.274
days active		0.054	tenure days		-0.273
max stake		0.050	site ABS		-0.241
hands w1		0.035	active day share		-0.232
min stake		0.034	days active		-0.224
vpip pfr gap		0.034	distinct stakes		-0.203

Recency does most of the work — GBT importance 0.249, and the next feature is worth a third as much. The behavioural columns, which carry the whole ecosystem thesis, add real but **secondary** signal. An examiner will ask; the ablation in §5.6 is the answer, and it is a decent one, but we do not oversell VPIP.

6.3 Model 2 • Unsupervised — what kind of player is leaving

K-MEANS AGAINST BISECTING K-MEANS, ON PRIOR-WINDOW BEHAVIOUR

The classifier says *who* is drifting. It cannot say *what kind of player* they are — and that is the entire thesis of this engagement, because an operator earns rake only while recreational players keep sitting down. This is the layer that connects them.

Clustered on the prior-window columns, not the lifetime ones. Lifetime rates average over the week being predicted — a lapsed player contributes nothing to their own final week and an active one does — so segments built that way partly encode the lapse answer before the cross-tab is drawn. Clustering the p_ columns means **every segment is knowable on the cutoff day**, which is also the only version an operator could act on.

SEGMENT	SHARE	HANDS	TABLES	VPIP	PFR	BB/100	LAPSE RATE	OF ALL LAPSERS
The Grinder	3.7% 2,831	8,799	11.5	0.18	0.12	-3.3	61.3%	3.2%
The Regular	35.4% 27,376	884	3.0	0.22	0.11	-7.2	61.6%	30.9%
The Gambler	19.8% 15,262	331	1.3	0.59	0.20	-31.8	74.6%	20.9%
The Recreational	41.2% 31,799	352	1.4	0.39	0.11	-14.3	77.1%	45.0%

K-means, k=4, five venues, 77,268 players, clustered on prior-window behaviour. **Names are interpretation; every figure is measured.** Nothing about leaving was in the feature set — the lapse column was cross-tabbed afterwards.

THE FINDING — THE ECOSYSTEM THESIS, MEASURED RATHER THAN ASSUMED

The two segments that lose money fastest are the two that leave fastest. The Gambler bleeds -31.8 bb/100 and lapses at 74.6%; the Recreational player loses -14.3 and lapses at 77.1%. The Grinder, who loses least (-3.3 — essentially just the rake), is the **stickiest player on the platform**. Together the two loose, money-losing segments are **60.9%** of players and **65.9%** of everyone who goes quiet. The operator's problem is not "we lose players". It is **"we lose the players we earn from, and keep the ones we don't."**

The archetype check — two confirmed, two wrong, and the wrongness is the interesting part

Before clustering we wrote down four archetypes from community rules of thumb, explicitly as hypotheses rather than findings. Matching the clusters against them by VPIP, PFR and table count:

HYPOTHESIS	WHAT WAS FOUND	VERDICT
Grinder — VPIP 22 / PFR 18 / 12 tables	VPIP 18 / PFR 12 / 11.5 tables	Confirmed. The closest match of the four.
Recreational — VPIP 45 / PFR 8 / 1–2 tables	VPIP 39 / PFR 11 / 1.4 tables	Confirmed.
Nit — VPIP 12 / PFR 9 / 3–5 tables	nearest cluster is VPIP 22	Not found. No extreme-tight cluster exists in this population at all.
Gambler — VPIP 60 / PFR 35	VPIP 59 / PFR 20	Half right — and the miss is a real finding. The loose cluster is loose-passive, not loose-aggressive: it enters 59% of hands but raises pre-flop in only 20%, so it <i>calls</i> on roughly two thirds of the hands it plays. That is the profile of someone paying to see cards rather than attacking pots — and it is also the segment losing money fastest, which is exactly what that profile predicts.

We report the misses because they are evidence the clusters were not fitted to our expectations.

Why multi-tabling earns its place in the feature set

Two views were built. **STYLE** (all six venues) uses betting behaviour only, deliberately excluding volume so that "this segment lapses more" is a statement about behaviour rather than an arithmetic restatement of "this segment plays less". **ECOSYSTEM** (five venues) adds intensity and multi-tabling.

THE RESULT THAT JUSTIFIES THE SHOWPIECE FEATURE

In the style view, no professional cluster forms at all — two of the four clusters both map onto "Recreational". **Style cannot separate a professional from a customer; table count can.** That is a direct argument for the multi-tabling feature, and for why iPoker's missing table identity was worth nulling rather than filling with a plausible 1.

WHAT WE WILL NOT OVERSELL — THESE ARE GRADIENTS, NOT SPECIES

Silhouette peaks at **k=2** (0.439), and k=4 costs 0.12–0.15 of it. **k=4 was chosen for business readability and one-to-one archetype comparison — a judgement call, recorded on purpose, not a silhouette optimum.** The second required configuration, **bisecting K-means**, scores *better* in both views (0.319 against 0.228 here), and the two algorithms place only **67%** of players in the same group. **Two algorithms disagreeing on a third of players is what weak separation looks like from the inside.** These are four useful cuts through a continuum, and they should be presented that way to management — the business conclusion survives the instability, because it rests on the ordering of lapse rates, not on any player's cluster membership.

6.4 Model 3 · Attaching money — and the defect that hid in it

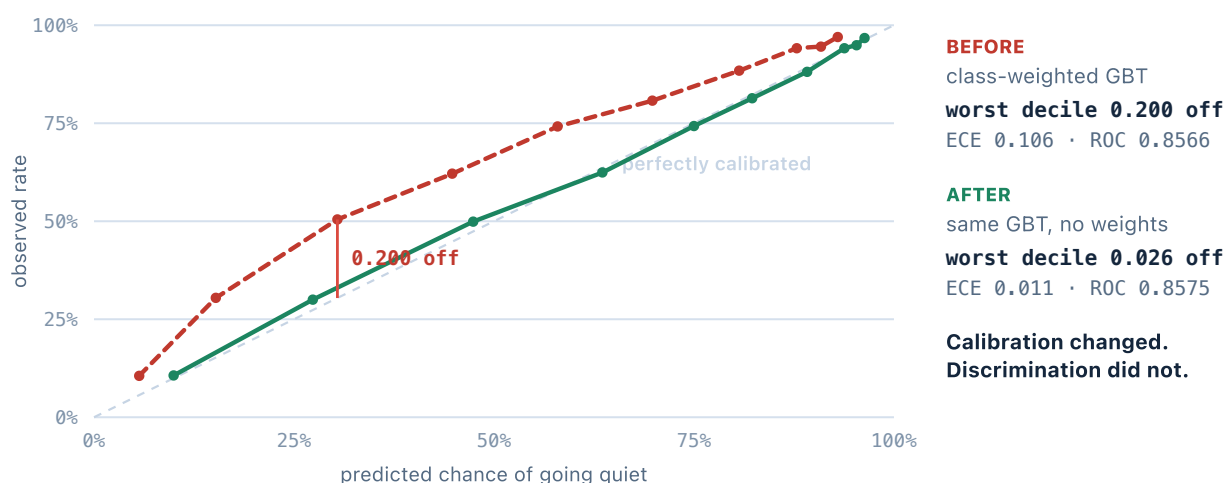
PROBABILITY CALIBRATION, THEN EXPECTED LOSS PER NAMED ACCOUNT

Expected rake at risk is one multiplication — **P(lapse) × weekly rake** — and **both factors had a defect that had to be closed before multiplying them meant anything**. The rake half is \$5.7. This is the probability half.

Ranked correctly, priced wrongly

The shipped classifier was fitted with balanced class weights. That drags every predicted probability toward 0.5 — which **ROC-AUC and PR-AUC cannot see, because they only read order**, and which a dollar figure notices immediately, because every dollar is multiplied by a number that is systematically too small.

The fix tried first was simply to remove the weights: at 68/32 there was never a real imbalance to correct. Weighting exists for problems where the positive class is 1% of the data; here **the majority class is the event of interest**. Same features, same split, same seed, same hyper-parameters — only `weightCol` removed.



Ten equal-sized deciles of predicted probability, each plotted against what actually happened. **The dashed red line is the shipped classifier**; the solid green line is the same model with class weighting removed. ROC-AUC moves by 0.0009 — inside gradient boosting's own reproducibility wobble.

CALIBRATION CHANGED; DISCRIMINATION DID NOT — WHICH IS EXACTLY THE RESULT TO WANT

The worst reliability decile moves from **0.200 off** to **0.026**, expected calibration error from 0.106 to 0.011, while ROC-AUC moves from 0.8566 to 0.8575 — inside gradient boosting's own reproducibility wobble. **A tolerance of 0.03 on the worst decile was written into the source before the run**, with isotonic regression (fitted on the train fold only) as the declared fallback. At 0.026 the refit cleared it, so **no calibration layer was fitted** — the probabilities are the model's own.

Calibration was then re-measured *per fold*, because three quarters of the players in the headline are ones the model trained on: **train fold mean predicted 0.682 against observed 0.682** (worst decile 0.035), **test fold 0.680 against 0.683** (worst decile 0.026). The means agree on both folds, which is what matters for a sum — the headline does not inherit an in-sample optimism — and the train fold is *not* the better-calibrated of the two, which it would be if the model were memorising.

WHAT THIS WAS WORTH — 30% OF THE HEADLINE

Using the original uncalibrated scores would have put the rake at risk at **\$424,810** instead of **\$604,163** — **29.7% low**. A business case understated by nearly a third, from a defect that *no ranking metric in the standard toolkit can detect*. One consequence to carry forward: the tuned threshold of 0.25 belonged to the weighted scores and **does not transfer**. The recalibrated model's best-F1 operating point is **0.40**, giving F1 **0.867** — a shade better than the tuned weighted model, at a threshold that now means what it says.

The output: one row per player, with a dollar figure on it

The shipped table `gold/rake_at_risk` carries, for each of **77,268** players: their segment, their calibrated risk score, their weekly rake split into measured and estimated, the measured share, and the product — **expected rake at risk**. That table is what the dashboard reads, what the deck quotes, and what a retention team would actually receive.

One reproducibility note for anyone re-deriving these figures. `risk_cal` is stored rounded to four decimal places while `expected_rake_at_risk` was computed from the unrounded probability, so multiplying the two shipped columns back together differs by up to \$0.16 on a player — **use the stored column**. For the same reason, counting players at `risk_cal ≥ 0.5` in the published table returns **58,837**, one more than the 58,836 computed in memory during the run. The difference moves no dollar figure, and both numbers are stated rather than quietly reconciled.

6.5 Model evaluation — which metric, and why

Different metrics answer different questions, and quoting the wrong one is how a weak model gets through review. Each was chosen for a stated reason:

METRIC	WHAT IT ANSWERS	WHY IT IS (OR IS NOT) QUOTED HERE
ROC-AUC	"Does the model rank a lapser above a non-lapser?"	The headline metric. Prevalence-independent, so it is the only number comparable across venues at 40–78% prevalence and across both populations.
PR-AUC	Ranking quality concentrated on the positive class	Reported, but moves with prevalence , so never compared across populations. Its floor here is the prevalence (0.683), not 0.5.
F1 at a chosen threshold	"How good is the deployed rule?"	Reported at 0.5 <i>and</i> at the tuned point, because the difference is the argument.
Precision at a contact budget	"Of the calls we can actually make, how many land?"	The metric the business runs on — the only one that respects the real constraint, which is capacity.
Reliability / ECE	"Does a score of 0.7 mean 70%?"	Essential the moment a probability is multiplied by money , and invisible to every metric above it. Worth 30% of the headline (\$6.4).
Accuracy	"What share of predictions are right?"	Deliberately not used. At 68% prevalence, "everyone lapses" scores 68% accuracy while being useless.
Silhouette	Cluster separation	Reported for $k=2\dots 8$ and for both algorithms, including where it argues against our chosen k.

6.6 Final model selection, justified

SHIPPED

Gradient-boosted trees, CORE feature set, threshold 0.40, no class weights

ROC-AUC **0.8575** · F1 **0.867** · worst reliability decile **0.026** · all six venues.

- Ties random forest on ranking and beats it on nothing — **chosen on calibration and on having the smaller, better-behaved probability surface**, not on ROC-AUC.
- CORE over EXTENDED because +0.006 ROC-AUC does not justify dropping a venue.
- Unweighted because the calibration test said so, with the tolerance declared in advance.

KEPT ALONGSIDE, NOT DISCARDED

Logistic regression · random forest · bisecting K-means

All three are in the shipped code and all three are reported. The linear model is the one to reach for if a decision has to be explained to a customer or a regulator: it costs 0.019 of ROC-AUC and returns a signed coefficient per feature. **A model you can defend at 0.837 may be worth more than one you cannot at 0.857**, and that is a business judgement rather than a statistical one.

6.7 Where these models are weak

Stated by us, in the body, rather than left in a footnote.

LIMITATION	WHY IT MATTERS, AND WHAT WOULD CLOSE IT
No temporal backtest	The split is by <i>player</i> , so no player is in both folds — but nothing is tested on a <i>different period</i> . We claim no temporal generalisation. The obvious check, train at <code>site_last_day - 14</code> and test at <code>- 7</code> , has not been run.
Recency dominates	The strongest feature is one a spreadsheet already has, and the free baseline scores 0.800. The model's value is concentrated in <i>ordering the top of the list</i> , which is what the contact-budget table measures.
Weakest where lapsing is rarest	Ongame 0.769 and Absolute 0.780 against PokerStars' 0.861. Small test folds explain part of it, not all — a direct argument for recommendation 3.
Clusters are unstable	Two algorithms agree on 67% of players. Use segments to allocate budget, never to justify an action against one named account.
GBT is not bit-reproducible	Spark sums tree histograms in task-completion order, so two runs of identical code differ around the sixth decimal (0.856638 vs 0.856641). Logistic regression and random forest reproduce exactly. We quote three decimals and do not chase the wobble.
One cutoff, one week	Every figure describes one seven-day window per venue. Nothing establishes that the <i>size</i> of the opportunity is stable week to week — only that the method finds it.

7 Business Insights

FIVE FINDINGS, EACH WITH A NUMBER AND A CONSEQUENCE ATTACHED

7.1 The headline

\$604,163 of next week's rake sits at risk across **77,268** players on five venues — **30%** of it recreational — and contacting the top 10% by expected loss (**7,727** players) puts **\$421,064** of it in reach.

Weekly rake **\$2,114,626**, **34.4%** measured · **58,836** players (76.1%) are more likely than not to play zero hands next week, holding **\$498,156** of weekly rake · iPoker excluded · 2009 US dollars, no conversion, no inflation adjustment.

7.2 Insight 1 — we lose the players we earn from, and keep the ones we don't

This is the ecosystem thesis, and it stops being an assumption the moment the segment table is cross-tabbed against lapse. Ranked by the money at stake rather than by headcount:

SEGMENT	PLAYERS	WEEKLY RAKE	PER PLAYER	LAPSE RATE	\$ AT RISK	SHARE OF RISK	MEASURED
The Recreational	31,799	\$432,177	\$13.59	77.1%	\$182,482	 30.2%	37.7%
The Gambler	15,262	\$380,162	\$24.91	74.6%	\$157,897	 26.1%	29.2%
The Regular	27,376	\$676,975	\$24.73	61.6%	\$155,173	 25.7%	33.1%
The Grinder	2,831	\$625,312	\$220.88	61.3%	\$108,610	 18.0%	36.7%
All five venues	77,268	\$2,114,626	—	70.5%	\$604,163	100%	34.4%

Weekly rake is the last seven days of each venue's own prior window — the same window the label is measured over, so both halves of the multiplication describe the same period. Rows shaded red are the loose, money-losing segments.

The recreational segment is **41.2% of players and carries 30.2% of all the rake at risk** — the largest single block — because it is the segment that leaves fastest. The Gambler adds another 26.1%. Together, those two loose, money-losing segments hold **56%** of the rake at risk (**\$340,379**) while the professionals hold 18.0%.

WHY THIS REFRAMES THE RETENTION BUDGET

A conventional retention programme is built around **value tiers**: protect the biggest spenders first. On a pure contributed-rake basis that means protecting the Grinder, who generates **\$220.88** of weekly rake per head against a Recreational player's **\$13.59** — **16** times as much. The segmentation says that is precisely backwards, and §7.3 explains why the two statements are both true at once.

The measurement behind “the money-losing segments are the ones that leave”

It is worth being precise about what has and has not been shown here. The clustering used **no information about leaving** — the lapse column was not in the feature set and was cross-tabbed afterwards. So the association between *losing money fast* and *going quiet* is an observed property of the population, not something the algorithm was steered toward. What has not been shown is causation: this data cannot separate “they left because they lost” from “the players who play loosely also happen to be the least committed”. Both readings support the same action, which is why the recommendation stands on the association alone.

7.3 Insight 2 — who **paid** the rake is not whose money **funded** it

Per head, a Grinder is worth sixteen times a Recreational player. Per segment, the Recreational and Gambler blocks hold three times the professionals' exposure. Read carelessly, those two facts look like a contradiction, and an examiner will go straight at it. They are not, and the distinction is the single most important idea in this report.

Contributed rake measures who put the money into the pot. It does not measure where that money came from.

A grinder pays a great deal of rake — with money that was, in the main, lost to them by recreational players. The profit rates measured in §6.3 say exactly that: the Gambler runs at **-31.8** bb/100 and the Recreational player at **-14.3**, while the Grinder sits at **-3.3** — essentially break-even before the rake. **Kill the recreational supply and the grinder's contribution does not survive it**, because there is nothing left to recycle. The professional is not a customer generating \$220 a week; he is a *conduit* for \$220 a week of somebody else's deposits.

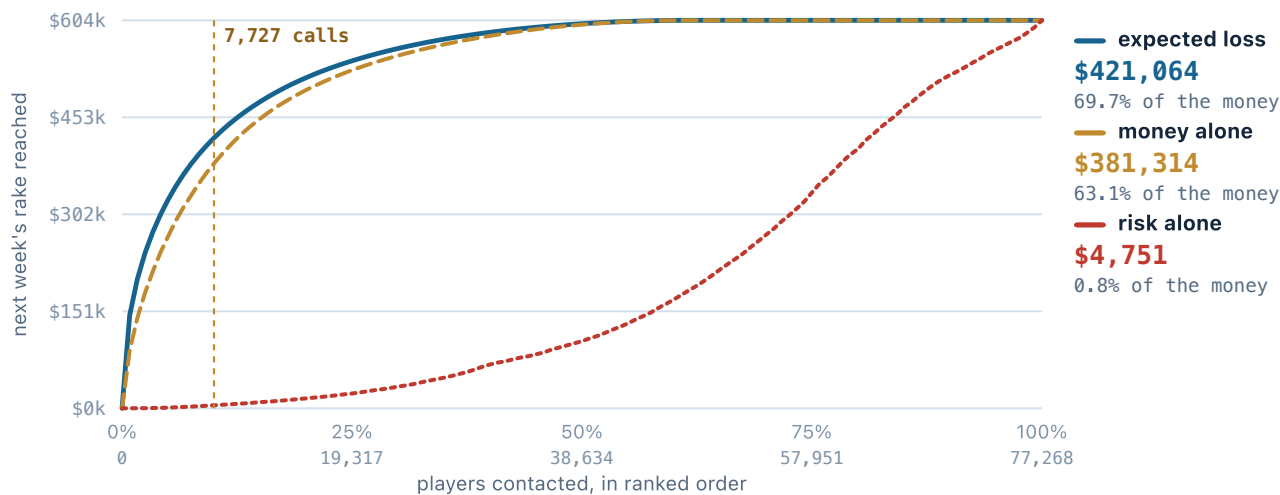
What has not been measured, said plainly: a “funded-by” attribution — allocating each hand’s rake in proportion to *net losses* rather than to contributions — would quantify this directly, and it has not been run. Contributed rake is one attribution among several and it is the industry standard; we use it because it is defensible and reproducible, and we state its limit rather than letting the ecosystem story quietly borrow authority it has not earned. It is the first item on the analytical roadmap in §8.3.

7.4 Insight 3 — a retention list sorted by churn risk is almost exactly the wrong list

This is the most useful operational result in the engagement, and it is a finding about the **ranking rule**, not about the model. The same 77,268 players, ranked three ways, top 10% contacted:

SORT THE LIST BY	RAKE REACHED	SHARE OF THE MONEY	TRUE LEAVERS	PRECISION	NAMES SHARED WITH ROW 1	HELD-OUT FOLD
Expected loss calibrated risk x weekly rake	\$421,064	69.7%	4,146	54%	100.0%	70.0%
Money alone rank by weekly rake	\$381,314	63.1%	2,521	33%	67.4%	63.7%
Risk alone rank by the churn score	\$4,729	0.8%	7,554	98%	0.1%	0.9%

The same 77,268 players, 7,727 calls (10%), three ways of choosing who gets them. The last column repeats the exercise on the 1,936 held-out players alone — labels no model ever trained on — and the picture is identical.



Each curve recomputed from the shipped Gold table by `src/report_figures.py`, which checks itself against the model run’s own JSON before writing. **One figure moves by design:** the risk-only ranking re-derives to **\$4,751** here against **\$4,729** in the run log, because `risk_cal` ships rounded to four decimals and its many ties break differently outside the run. It is 0.8% of the money either way, and it is stated rather than hidden. Series carry dash patterns as well as colour, because red and gold sit inside the colour-vision confusion band.

THE RESULT IN ONE LINE, WITH THE UNCOMFORTABLE PART LEFT IN

Ranking by risk alone and ranking by expected loss produce lists that share **0.1%** of their names. **The risk-only list is better at its stated job** — it finds 7,554 genuine leavers against 4,146, at 98% precision — and it reaches **0.8%** of the money. **It is a list of certain departures worth nothing.** Sorting by money alone is much closer to right (63.1% against 69.7%) but wastes two thirds of its calls on players who were not going anywhere. **The product of the two beats both.**

CHECKED ON PLAYERS NO MODEL EVER SAW

Repeated on the held-out fold alone — 19,360 players, 1,936 calls, labels never trained on — the picture is identical: **70.0%** of the money against **0.9%**, on **0.2%** shared names. **The recommendation does not depend on players the model had already met.**

7.5 Insight 4 — the five venues are five different businesses

A pooled figure describes none of them. Lapse rates run 40.4%–77.9%, weekly rake per player \$18.99–\$103.82, and evidence quality 11%–97% measured:

VENUE	PLAYERS	WEEKLY RAKE	PER PLAYER	LAPSE RATE	\$ AT RISK	SHARE OF RISK	MEASURED
PokerStars	48,561	\$922,104	\$18.99	77.9%	\$270,726	44.8%	15.5%
PartyPoker	12,976	\$488,652	\$37.66	73.8%	\$184,389	30.5%	33.3%
Ongame	3,722	\$386,417	\$103.82	49.8%	\$77,486	12.8%	84.7%
Absolute	6,737	\$206,273	\$30.62	45.8%	\$46,966	7.8%	33.3%
Full Tilt	5,272	\$111,180	\$21.09	40.4%	\$24,596	4.1%	23.4%

Five venues; iPoker excluded. **Per-player rake differs by a factor of five mostly because the venues sit at different stakes**, not because their players behave differently.

Two things a pooled number hides. **PokerStars is 63% of the players and 45% of the rake at risk**, so the headline is substantially a PokerStars statement — and it has the *lowest* measured share (15.5%), because its reconciling hands are its smaller pots. And per-player rake differs fivefold because the venues sit at different stakes: Ongame’s hands are almost all at a \$4–\$10 big blind, PokerStars’ are 53% at \$0.25. **A pooled list over-serves one venue and under-serves four.**

7.6 Insight 5 — two thirds of the revenue line is not recorded anywhere

The last finding is not about players at all, and it is the one with the longest payback.

A GOVERNANCE FINDING THAT FELL OUT OF AN ANALYTICS PROJECT

The operator’s own income had to be **reconstructed from the pot up**, and only **34.4%** of it reconciles exactly; one venue exports nothing usable on any of its 5,996,741 hands. **Every downstream number inherits that** — the estimator in §5.7 makes the reconstruction defensible, it does not make it a measurement. **One logged column per hand would retire the largest caveat in this engagement**, and would do it for every future analysis too.

7.7 The decision surface

Insights that live in a PDF get read once. The shipped artifact is docs/dashboard.html — one self-contained file, no server and no internet, carrying all 77,268 Gold rows encoded in the page and **recomputing every headline, curve and ranking in the browser** from the filter you set. Because the whole population travels with the page rather than a pre-aggregated cube, **any** venue × segment × stake combination is exact rather than interpolated.

WHAT IT OPENS ON

The headline sentence, filter-aware — then the contact-budget panel with the three ranking curves, the segment × venue money matrix you click to drill, and the call list with CSV export. **A decision, not a chart.**

THE TRUST STRIP

Measured-versus-modelled coverage, the reliability curve, model against the free recency baseline, each venue’s own clock, the iPoker exclusion and the four sensitivities — **on the same page as the number**, not in an appendix nobody opens.

7.8 How much of the headline is a decision rather than a measurement?

Four decisions in the chain could have gone another way. Each was re-run end to end:

DECISION	HEADLINE UNDER THE ALTERNATIVE	READING
Which probability the uncalibrated weighted scores	\$424,810 — 29.7% lower	Moves the headline by nearly a third. This is the entire reason §6.4 exists.
Rake estimator the naive per-player rate × hand count	\$2,483,876 vs \$1,981,801 of weekly rake — 25% high	Directionally exactly as §5.7 predicts, on the 70,866 players it can be computed for.
Attribution rule equal split between everyone dealt in	prior-window rake \$5,569,696 vs \$5,682,448, ratio 0.980	Does not move the headline. The rule shifts ~2% of the money between players.
Modelled venues the two best-covered venues only	21.0% of their weekly rake at risk vs 28.6% pooled	Tracks lapse prevalence, not the estimator : these two venues (67% measured) have the lowest lapse rates.

The last row needs explaining rather than alarm: the gap tracks **lapse prevalence, not the estimator**. Ongame and Absolute have the lowest lapse rates in the dataset (49.8% and 45.8% against PokerStars’ 77.9%), so less of their rake is at risk for reasons unrelated to pricing. **The attribution rule does not move the headline; calibration moves it by nearly a third.**

"Improve retention" is not a recommendation. Each of the four below names the action, the money attached to it, who owns it, how to measure it, and what would make it the wrong call.

01 Rank Monday's call list by expected loss, not by churn risk

Change one ORDER BY clause in the retention workflow: sort by **calibrated risk x weekly rake**, not by risk. The same 7,727 calls then reach **\$421,064** of next week's rake instead of **\$4,751**. It costs nothing — no extra contacts, no new budget, no new system — and it is the single highest-return action in this report.

OWNER	Retention & CRM lead	MEASURE	Rake retained from contacted accounts vs a held-back control cell, weekly
--------------	---------------------------------	----------------	---

RISK	The ranking is validated on held-out players but never on a <i>later period</i> . Run it as an A/B against the current list for four weeks before retiring the old one.
-------------	---

02 Move retention spend from the top spender to the recreational seat

The Recreational and Gambler segments hold **56%** of the rake at risk (**\$340,379**) against the professionals' 18.0% — and they are the money the professionals are paid from (\$7.3). Concretely: shift a defined slice of the VIP budget into **low-value, high-frequency interventions** aimed at accounts losing faster than -20 bb/100 — deposit-matched small bonuses, softer table recommendations, and a stake-down prompt rather than a re-deposit prompt.

OWNER	VIP & promotions	MEASURE	Recreational share of rake at risk (KPI 3) falling while total liquidity holds
--------------	-----------------------------	----------------	--

RISK	Segments are gradients, not species — two algorithms agree on only 67% of players. Use segments to allocate budget across groups , never to justify treating one named account differently.
-------------	--

03 Run the list per venue, never pooled — and give each venue its own clock

Lapse rates span 40.4% to 77.9% and money coverage 11% to 97%. One venue is 45% of the exposure, so a pooled average describes none of them and over-serves that one. The same rule applies to the *window*: "quiet for a week" must be measured against each venue's own last activity, not a global calendar — doing otherwise mislabels 44% of the population by construction.

OWNER	Regional operations	MEASURE	Per-venue precision at budget, reported separately — never a single pooled number
--------------	----------------------------	----------------	---

RISK	Model quality is <i>lowest</i> where lapsing is rarest (Ongame 0.769), so the smallest venues need the free recency baseline retained as a comparison for longer.
-------------	---

04 Record rake at source, and this stops being an estimate

Two thirds of the weekly rake in this report is modelled because **no venue writes it down**. One logged column per hand — rake charged, in the pot's own currency — would retire the single largest caveat in the engagement, remove the plausibility screen, the estimator and its 0.7% error bar entirely, and improve every future analysis rather than just this one. It is the cheapest item on this page and the one with the longest payback.

OWNER	Data engineering	MEASURE	KPI 4 — measured share of revenue, from 34.4% toward 100%
--------------	-------------------------	----------------	---

RISK	None analytically. The cost is an export-schema change per venue, which is a vendor negotiation rather than an engineering problem.
-------------	---

8.1 What this is worth, and how confident we are in it

The honest way to size the opportunity is as a range, because two of the four sensitivities in §7.8 move the number and two do not.

SCENARIO	WEEKLY RAKE AT RISK	WHAT DRIVES IT
Conservative — uncalibrated probabilities	\$424,810	The figure we would have reported without the calibration work in §6.4. Included to show the size of that one decision, not as a plausible alternative — it is <i>known</i> to be miscalibrated.
Reported	\$604,163	Calibrated probability × estimator-priced rake, contributed attribution. The number every other figure in this report is consistent with.
Reachable this week	\$421,064	The number to plan against. It assumes a 10% contact budget and no more; it does not assume any contact succeeds.

WHAT WE ARE EXPLICITLY NOT CLAIMING

Reaching a player is not saving one. Nothing in this data measures **intervention effectiveness** — there is no campaign history, no control group and no response label anywhere in the archive. The **\$421,064** is **revenue put within reach of a conversation**, not revenue recovered. Establishing what share is actually recoverable requires a held-back control cell, which is why it is step one of the roadmap below rather than an assumption in the headline.

8.2 Risks to the recommendations

RISK	SEVERITY	MITIGATION ALREADY IN PLACE, OR REQUIRED
The rake estimate is wrong in a way the validation cannot see	high	The estimator validates within 0.7% per venue — but only on hands that <i>reconcile</i> , which is the population it was fitted on. It conditions on everything recorded for every hand and cannot condition on what the data never records. Mitigation: recommendation 4.
No temporal validation	high	Nothing is tested on a different period. Run the A/B in recommendation 1 as a live temporal test before scaling; a backtest at <code>site_last_day - 14</code> is the offline equivalent.
Acting on a segment as if it were a fact	medium	Clusters are gradients. Budget allocation across segments is defensible; account-level treatment justified by cluster membership is not. Stated in recommendation 2.
Responsible-gaming exposure	medium	The list is, by construction, the accounts losing fastest and disengaging quietest . Any deployment must pair the bonus offer with a stake-down or cool-off path, and the same score must never be used to increase exposure. Non-negotiable, and cheap.
Model drift	low	Recency dominates and recency does not go stale. Retrain monthly; monitor the free recency baseline alongside the model so a divergence is visible without a label.

8.3 A 90-day roadmap

HORIZON	DO	WHY NOW
Weeks 1–2	Ship the expected-loss ranking as an A/B against the current list, with a held-back control cell	Costs nothing, needs no new data, and is the only way to convert “in reach” into “recovered”.
Weeks 2–4	Log rake per hand at source on one venue	Proves the schema change end to end on the venue with the worst coverage, before negotiating it across all six.
Weeks 4–8	Run the funded-by rake attribution and the time-shifted backtest	The two named gaps in this report. The first quantifies §7.3 instead of arguing it; the second is the missing robustness check.
Weeks 6–10	Add the value model as a second stage on the lapse list	The bake-off’s runner-up, already built. “Which lapsers are worth a call” is the question recall-at-budget says actually matters.
Weeks 8–12	Move ingest to a streaming topic per venue and add an orchestration layer	Makes recency current to the minute rather than the day — the strongest feature in the model — and retires the manual six-script run order.

Appendix A — References

Dataset

1. **Kim, Juho.** "Recording and Describing Poker Hands." *IEEE Conference on Games (CoG)*, 2024. DOI 10.1109/CoG60054.2024.10645611. — the peer-reviewed description of the PHH format used by every file in this project.
2. **University of Toronto Computer Poker Student Group.** *phh-dataset* (v3), 16 September 2025. Zenodo. DOI 10.5281/zenodo.17136841. Licence **CC BY 4.0**; the accompanying code repository is MIT-licensed. Archive size 20,289,230,983 bytes; the subset used is data/handhq only (21,782 files, 16.3 GB).
3. Repository: github.com/uoftcprg/phh-dataset. Acquired via `git clone --filter=blob:none --sparse` followed by `git sparse-checkout set data/handhq`.

LICENCE COMPLIANCE

CC BY 4.0 requires attribution, permits commercial use and permits redistribution of derivatives. Attribution is given above and in the deck's closing slide. The raw data itself is **not redistributed** in this repository — data/ is gitignored and only its folder structure is committed, with data/README.md documenting both download routes so any reader can reproduce the lake from the original source. The 619 M-hand Annual Computer Poker Competition portion of the archive (bots playing bots) was deliberately excluded as commercially meaningless.

Software

TOOL	VERSION	USED FOR
Apache Spark / PySpark	4.2	All distributed feature engineering and every shipped model — MLlib GBTCClassifier, RandomForestClassifier, LogisticRegression, KMeans, BisectingKMeans, Imputer, StandardScaler, VectorAssembler
Polars	1.43.2	Per-file parsing into Silver, and the dashboard, deck and report payloads — chosen where the unit of work fits in memory and Spark's overhead is pure cost
Python	3.13	Everything. tomllib (stdlib, 3.11+) parses the .phhs files directly — no third-party or regex parser was needed
scikit-learn	1.x	The Step-5 bake-off only — deliberately throwaway prototyping to choose the question before building it properly in MLlib. No shipped model uses it.
OpenJDK	17	The JVM Spark runs on
Google Chrome (headless)	—	Rendering this report and the deck to PDF

Domain references

4. VPIP / PFR reference ranges used as **hypotheses** for the archetype comparison in §6.3 are widely circulated poker-community rules of thumb, not findings from this dataset, and are labelled as such wherever they appear. The comparison in §6.3 reports two of them as **wrong**.
5. "No flop, no drop" and the standard 5%-capped rake structure are industry conventions of the period; both are independently **confirmed by measurement** in §2.1 and §5.7 rather than assumed.

Project documents (in this repository)

docs/rake-at-risk-results.md	The money layer, its four sensitivities and its limits — source for §5.7, §6.4, §7
docs/fork-a-results.md	The lapse model, its three baselines and its known defects — source for §6.2
docs/segments-results.md	The segmentation, the archetype check and the instability — source for §6.3
docs/bakeoff-decision.md	The five candidate questions and why Fork A won — source for §6.1
docs/poker-explained.html	Companion: poker from zero, game concept → data column
docs/data-pipeline-guide.html	Companion: the pipeline and Spark from first principles

Appendix B — AI-usage disclosure

Generative AI was used substantially in this project. This section states where, how it was checked, and what it was not allowed to do.

WHERE AI WAS USED	EXTENT	HOW THE OUTPUT WAS VERIFIED
Writing and debugging the Python/PySpark code in src/	substantial	Every script was executed against the real 21.6 M-hand dataset and judged on what it printed. Correctness is enforced by business identities rather than code assertions — the zero-sum rake identity, the big-blind VIP check, the reconciliation, the attribution cross-check — all shown in §5 with their measured values. Several AI-written first attempts were wrong and were caught by exactly these tests (§5.3 and §5.7).
Drafting this report, the deck and the guides	substantial	Structure and prose were drafted with AI assistance and edited by the team. No figure in any of these documents was written by a language model : every number is injected from the model runs' own JSON by src/report_figures.py, deck_charts.py and dashboard_data.py, each of which refuses to write if its numbers disagree with the run log.
Explaining unfamiliar domain concepts (poker mechanics, Spark internals)	moderate	Used as a tutor, then checked against the data. A rule of thumb that could not be confirmed from the dataset is labelled as one — and §6.3 reports two of them as measurably wrong.
Interpretation, model selection and business judgement	reviewed	AI proposed candidate analyses; the choices recorded in §6.1, §6.6 and §8 were made and are owned by the team. Every judgement call in this report is labelled as one — k=4, the money_hands ≥ 50 threshold, the 6% plausibility ceiling, CORE over EXTENDED.

THE THREE RULES WE HELD IT TO

1. **No number enters a document except by being computed.** Figures are injected from JSON by script, and the scripts self-check against the run logs before writing. This is verifiable by anyone: run src/report_figures.py --check and it exits non-zero if a single figure has drifted.

2. **Nothing is claimed that was not run.** Where an analysis was *not* done — the temporal backtest, the funded-by attribution, intervention effectiveness — it is named as not done, in the body (§6.7, §7.3, §8.1).

3. **The tests are business statements.** A check that a domain expert can validate without reading Python is worth more than one that only a programmer can. That is why “the operator’s take must equal the players’ combined loss” is the load-bearing test in this pipeline.

Tools used: Claude (Anthropic) via Claude Code, for code authoring, debugging, drafting and explanation. No AI service was given the dataset to analyse independently — all computation ran locally on the machine in Appendix E, reproducible from src/ at seed 42.

Appendix C — Team contributions

TO BE COMPLETED AND SIGNED OFF BEFORE SUBMISSION

The required contribution record. **Fill in names, roll numbers and the workstream each member owned**; the workstream column is pre-filled from the real structure of the work, so the split is recorded against actual deliverables rather than invented afterwards.

TEAM: complete this table

NAME	ROLL NO.	WORKSTREAM OWNED	ARTIFACTS
—	—	Data acquisition & Bronze/Silver build	src/parse_phh.py · src/build_silver.py · §3, §5.1–5.3
—	—	Feature engineering & Gold	src/features.py · src/features_lapse.py · §5.4–5.6
—	—	Supervised modelling & evaluation	src/bakeoff.py · src/fork_a.py · §6.1–6.2, §6.5–6.7
—	—	Segmentation & the money layer	src/segments.py · src/rake_at_risk.py · §6.3–6.4, §5.7
—	—	Decision surface & visual design	docs/dashboard.html · src/dashboard_data.py · §7.7
—	—	Narrative, report & deck	this report · ecosystem-engine-deck.html · src/report_figures.py, deck_charts.py

Maximum six members per the brief. Where two members shared a workstream, list both rows against the same artifacts.

Appendix D — Learning outcomes mapped to evidence

The brief names eight learning outcomes across two course outcomes. Each row below points at the specific artifact that demonstrates it, so nothing has to be taken on trust.

OUTCOME	WHERE IT IS DEMONSTRATED	THE EVIDENCE, SPECIFICALLY
CLO 3 • BUSINESS ANALYTICS		
Translate an ambiguous business problem into an analytical problem	§2.4 · §6.1	“Our ecosystem is unhealthy” narrowed in three recorded steps into a label the data can carry, after five candidate questions were prototyped and scored against their own baselines and four were rejected — one of them, hand economics, on a measured skill of 0.004.
Identify meaningful KPIs	§2.5	Four KPIs, each readable off the shipped Gold table and each moving a budget: rake at risk, reach at a fixed contact budget, recreational share of exposure, and measured share of revenue — the last a governance KPI that came out of the engineering work.
Generate actionable recommendations	§8	Four decisions, each with a dollar figure, a named owner, a success measure and a stated risk. Recommendation 1 is a one-line change to an ORDER BY worth \$421,064 against \$4,751 .
Communicate insights suitable for senior management	§1 · the 10-slide deck · docs/dashboard.html	A one-page executive summary that stands alone, a deck where every slide title states its conclusion , and a self-contained interactive dashboard that opens on a decision rather than a chart and exports the week’s call list as CSV.
CLO 4 • BIG DATA ENGINEERING		
Build scalable data pipelines	§4 · §5.1–5.3	A Bronze/Silver/Gold medallion lake: 21,782 files → three partitioned Parquet tables → one player-grain table, with the parse reconciling to the published hand count exactly and every dropped hand counted with a reason.
Process large datasets using distributed computing	§4.4 · §5.6	PySpark 4.2 over 116,621,636 seat-rows and 183,671,936 action-rows — 111× and 175× Excel’s row limit — with four non-default Spark settings each fixing a documented failure.
Engineer features suitable for machine learning	§5.6	61 columns at player grain, all recomputed under day ≤ cutoff so they are knowable on the decision day; encoding, scaling and imputation each justified; two leaks found and proved — one by arithmetic, one by algebra — and a third tested by ablation.
Apply modern cloud-based analytics environments	§4.6	Written as PySpark against a partitioned Parquet lake, which is what a Databricks workspace runs on. The move is a re-hosting job — paths become dbfs:/, tables become Delta, local[8] becomes a cluster spec — and no analysis code changes. Databricks execution is deliberately deferred; the local Spark run is complete and measured.

RUBRIC LEDGER — WHERE EACH MARK IS EARNED

CRITERION	MARKS	PRIMARY EVIDENCE
Business problem & executive storytelling	3	§1, §2, §7 · the deck’s conclusion-first titles
Data engineering & architecture	4	§4, §5.1–5.5, §5.7 · the exact reconciliation · the zero-sum identity
Feature engineering & data preparation	3	§5.6 · the prior-window construction · two proved leaks · the null-never-impute policy
Machine learning implementation	4	§6.2–6.4 · three MLlib classifiers, two clustering algorithms, two feature sets
Model evaluation & interpretation	2	§6.5 · three baselines · the reliability curve · threshold as a decision · per-venue scores
Business recommendations & strategic value	2	§8 · four costed decisions with owners, measures, risks and a 90-day roadmap
Professionalism, documentation & presentation	2	This report · the repository’s commit history · a results note per model, each with its own weaknesses section

Appendix E — Reproducibility

Everything in this report can be regenerated from the scripts in `src/`. Seed **42** throughout; the train/test split is a hash of the player key, so it is identical on any machine.

The machine

Apple M4 · 10 cores (4P/6E) · 16 GB RAM · OpenJDK 17 · Python 3.13 · PySpark 4.2 · Polars 1.43.2 · Spark local[8], 9 GB driver, 96 shuffle partitions, scratch on the project disk.

Run order and wall clock

SCRIPT	RUNTIME	PRODUCES
<code>src/build_silver.py</code>	27.7 min	Bronze → Silver. All 21,782 files on 8 cores → three Parquet tables, 44 parts each. Prints the reconciliation.
<code>src/features.py</code>	4.6 min	Silver → gold/hand_features + gold/player_features. Prints the zero-sum identity.
<code>src/bakeoff.py</code>	41 s	The five candidate questions, scored against their own baselines → docs/bakeoff-results.json. Throwaway by design.
<code>src/features_lapse.py</code>	3.8 min	gold/player_lapse — every feature recomputed over day ≤ cutoff only. The leakage fix.
<code>src/fork_a.py</code>	5.0 min	Three MLlib classifiers × two feature sets × three baselines + the ablation → docs/fork-a-results.json, gold/lapse_scores. --quick gives a 1.3-min smoke test.
<code>src/segments.py</code>	2.0 min	K-means vs bisecting K-means, two views, k-sweep → docs/segments-results.json, gold/player_segments.
<code>src/rake_at_risk.py</code>	2.1–5.1 min	Recalibration, rake pricing, attribution, the headline → docs/rake-at-risk-results.json, gold/rake_at_risk. --rebuild-rake redoes the 116 M-row attribution.
<code>src/dashboard_data.py</code>	0.2 s	Injects the Gold payload into docs/dashboard.html. Polars only.
<code>src/pipeline_facts.py</code>	~20 s	Re-measures the lake → docs/pipeline-facts.json. Aborts if the reconciliation identity fails.
<code>src/report_figures.py</code>	~2 s	Injects every figure and chart into this report . --check verifies without writing and exits 1 on drift.
<code>src/deck_charts.py</code>	~2 s	The same, for the executive deck.

REPRODUCIBILITY, STATED EXACTLY

The money layer was **run three times end to end and every figure came back identical** — the only field that changed between runs was the wall clock. Logistic regression, random forest and both clustering algorithms reproduce bit-for-bit. **Gradient-boosted trees reproduce only to about five decimal places** (measured: 0.856638 vs 0.856641 across two runs of identical code), because Spark sums tree histograms in task-completion order. Nothing in this report is quoted past three decimals for that reason, and the wobble is recorded here so a future reader does not mistake it for a bug.

Known outstanding items, so they are not discovered as surprises

- `features.py` still writes `+inf` into `avg_stack_bb` for iPoker players. The guard exists in `features_lapse.py`; back-porting it and rebuilding the lifetime Gold table has not been done. **No shipped figure depends on that column.**
- The **time-shifted backtest** and the **funded-by rake attribution** are both specified and neither has been run. Named in §6.7, §7.3 and the roadmap.
- Databricks execution is deferred by team decision; the local PySpark run is complete.

Appendix F — Glossary

THE GAME AND THE BUSINESS

Rake	The operator's cut of a pot — a capped percentage, charged only once a flop is dealt. The entire revenue line.
No flop, no drop	The industry rule that a hand ending before the flop pays no rake. Confirmed here on 99.95% of such hands.
Big blind (bb)	The forced bet that sets the stake. All money is normalised to it, because \$50 at 25NL and at 100NL are not the same event.
25NL / 100NL	Stake labels — the buy-in cap in dollars. 25NL is a \$0.25 big blind.
bb/100	Profit rate: big blinds won per 100 hands. The standard measure of a player's results. The player's profit, not the operator's revenue.
VPIP	Voluntarily Put money In Pot — share of hands a player chose to play. High = loose/casual, low = tight/selective.
PFR	Pre-Flop Raise rate. VPIP high with PFR low is the signature of a <i>passive</i> player who calls to see cards.
Showdown / WTSD	Reaching the end of a hand with cards revealed. Only 22.4% of hands do, which is why 77.6% of hole cards are never seen.
Multi-tabling	Playing several tables at once. The one feature that separates a professional from a customer (§6.3).
Uncalled bet	A bet nobody matched. It must be returned to the bettor, not left in the pot — the bug in §5.3.

THE DATA AND THE METHOD

Medallion (Bronze / Silver / Gold)	Raw as published → typed and modelled → aggregated for analysis. Each boundary buys something specific (§4.1).
Parquet	Columnar file format. A query touching 2 of 20 columns reads 2 columns — and compresses 7.0× here.
TOML	The plain-text format the raw hands are written in. Types are inferred <i>per value</i> , which is defect 1 in §3.4.
Shuffle	The step where distributed data is redistributed across machines to group it. Where memory pressure lives, and the only stage that genuinely needs a cluster.
Leakage	A feature that contains information from the period being predicted. Two kinds were found and proved here (§5.6).
Prevalence	The share of the population that is positive — 68.2% here. PR-AUC moves with it; ROC-AUC does not.
ROC-AUC	The chance the model ranks a random lapser above a random non-lapser. 0.5 is a coin flip.
Calibration / ECE	Whether a predicted 0.7 means 70%. Invisible to ROC-AUC, and worth 30% of the headline here (§6.4).
Silhouette	How well separated clusters are. Peaks at k=2 here (0.439), which is why we call the segments gradients rather than species.
Contact budget	How many accounts the team can actually call. Precision at a fixed budget is the metric the business runs on.

CLOSING NOTE

This report describes an eight-week engagement on a seventeen-year-old public dataset, done on one laptop. The numbers are 2009 dollars at micro stakes and should be read as a scale, not a forecast. **What transfers is the method:** derive the revenue line when nobody records it, prove the features cannot see the future, calibrate the probability before you multiply money by it, and check the ranking rule as carefully as the model — because in this case the ranking rule was worth a hundred times more than the model was.